



北京邮电大学

Beijing University of Posts and Telecommunications

深度神经网络与计算机视觉任务

戚 琦

网络与交换技术国家重点实验室 网络智能研究中心 科研楼511

qiqi8266@bupt.edu.cn

13466759972

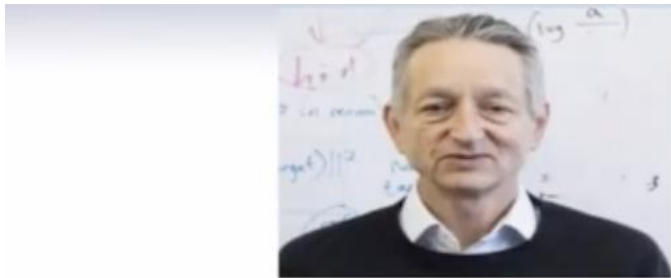


- 深度学习基本概述
- 卷积神经网络原理
- 常见的卷积神经网络结构
- 目标检测与分割

- 1958: 感知机 (linear model)
- 1980s: 多层感知机
与目前的全连接神经网络差别不大
- 1986: 反向传播
多于三层则用处不大
- 1989: 1 hidden layer is “good enough”, why deep?
- 2006: RBM initialization (breakthrough)
- 2009: GPU
- 2011: Start to be popular in speech recognition
- 2012: win ILSVRC image competition



对神经网络的坚持



Geoffrey Hinton



Yann LeCun



Yoshua Bengio

深度学习三剑客的坚持

LeCun: “这种算法很有价值，不知为什么要放弃它。”

Hinton: “智能产生于人脑，所以从长远来说，人工智能应该像大脑系统一样工作。”

这三人从2003年开始，一起组成了“Deep learning conspiracy”，悄悄开发了10层以上的人工神经网络。

2006年是深度学习的起始年，Hinton在Science上发表文章，提出一种叫做自动编码器（Auto-encoder）方法，部分解决了神经网络参数初始化问题。

> Science. 2006 Jul 28;313(5786):504-7. doi: 10.1126/science.1127647.

Reducing the dimensionality of data with neural networks

G E Hinton¹, R R Salakhutdinov

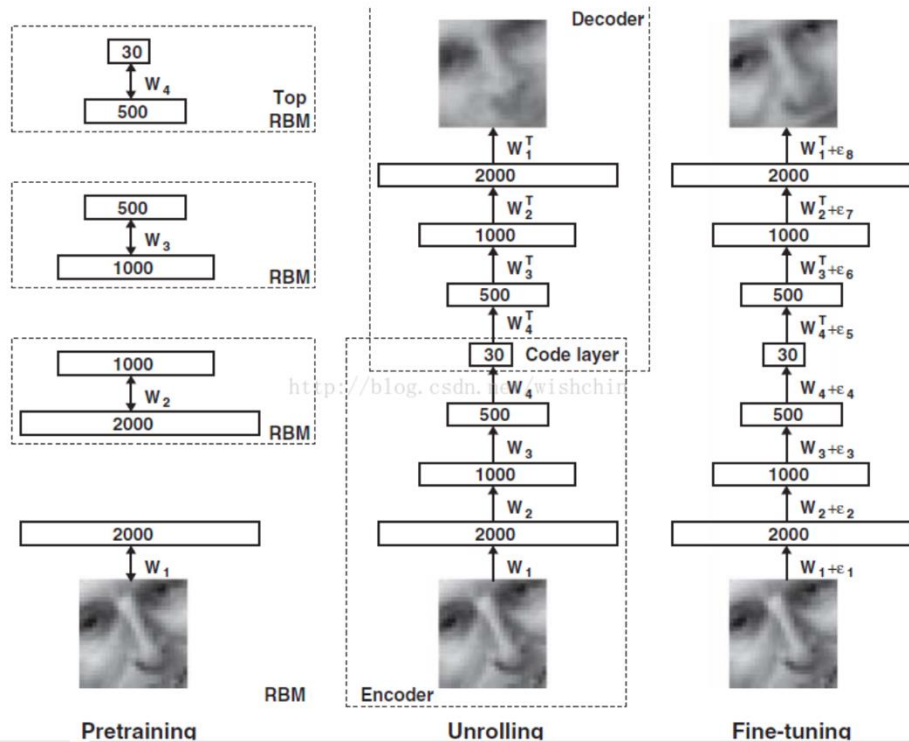
Affiliations + expand

PMID: 16873662 DOI: 10.1126/science.1127647

Free article

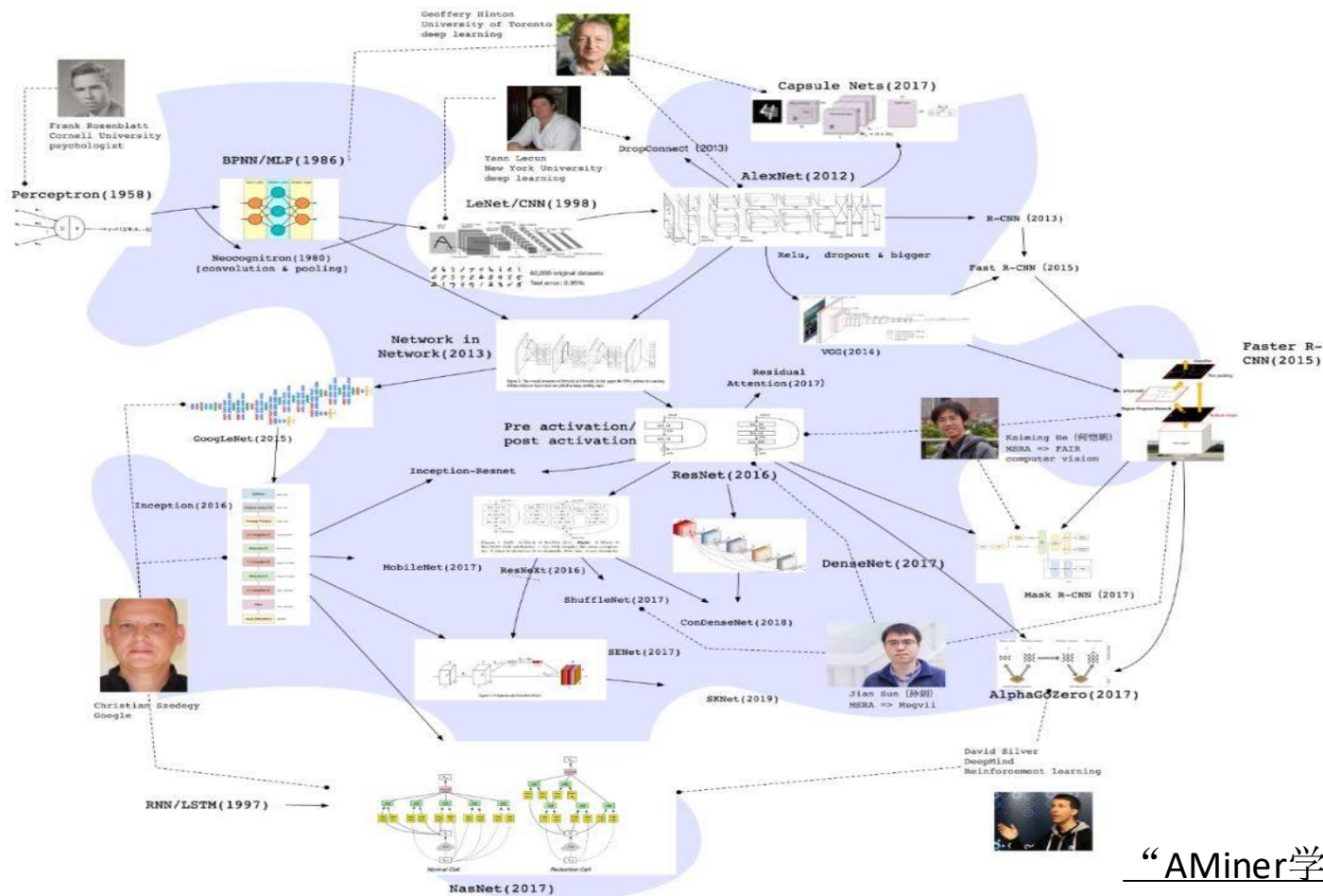
受限玻尔兹曼机栈（堆叠的RBM）

效果一般，但是开启了深度学习





卷积神经网络发展



1962年Receptive fields

- **1962年 Hubel和Wiesel:** 卷积神经网络的发展，最早可以追溯到1962年，Hubel和Wiesel对**猫大脑中的视觉系统的研究**。

在哈佛医学院建立了神经生物学系

Receptive fields, binocular interaction and functional architecture in the cat's visual cortex中提出了Receptive fields的概念。

记录了猫脑中各个神经元的电活动。使用幻灯机向猫展示特定的模式，并指出**特定的模式刺激了大脑特定部位的活动**。这种单神经元记录是当时的一项创新，由Hubel早期发明的特殊记录电极实现，通过这些实验系统地创建了视觉皮层的地图。



因其在视觉系统中信息处理方面的杰出贡献，他们在**1981年**获得了诺贝尔生理学或医学奖

论文: <https://www.aminer.cn/archive/receptive-fields-binocular-interaction-and-functional-architecture-in-the-cat-s-visual-cortex/55a5761e612c6b12ab1cc946>



- 1980年，日本科学家福岛邦彦提出了一个包含**卷积层、池化层的神经网络结构**。

Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position

获得 2021 年度鲍尔奖“通过发明第一个深度卷积神经网络「Neocognitron」将神经科学原理应用于工程的开创性研究，这是对人工智能发展的关键贡献”

上世纪80年也提出过**Attention**概念（注意力）和网络



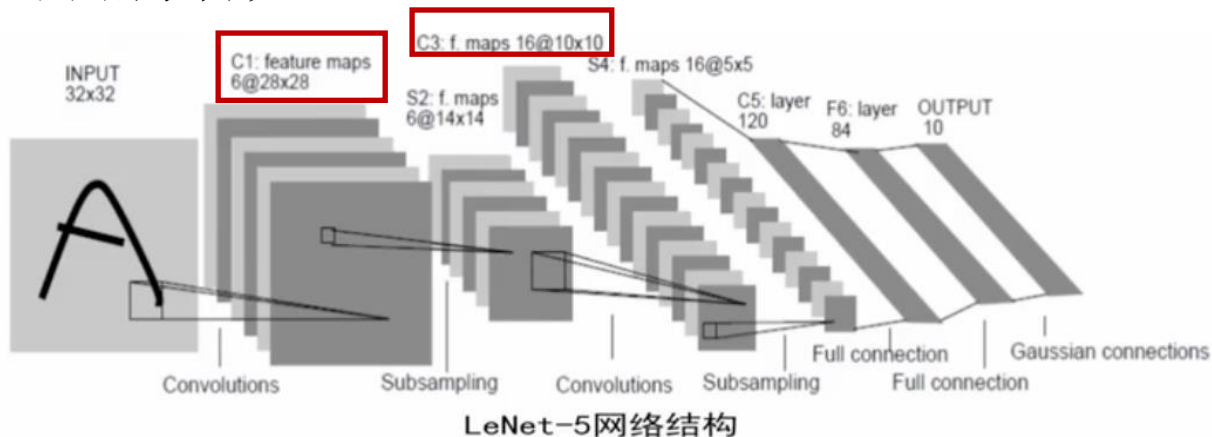
福岛邦彦

被誉为“八十多岁仍在奋斗的全球人工智能专家”。除了后来发展出卷积神经网络的Neurocognition（认知控制），现在深度学习中开始热闹起来的Attention（注意力）网络背后也有他的身影



1998年LeNet-5

- LeNet-5, 将BP算法应用到这个神经网络结构的训练上, 就形成了当代卷积神经网络的雏形



1. LeCun Y., Bottou L., Bengio Y., and Haffner P., Gradient-based learning applied to document recognition, Proceedings of the IEEE, pp. 1-7, 1998.



Yann Lecun

阅读支票、识别数字之类的任务上很有效果, 但由于在一般的实际任务中表现不如SVM、Boosting等算法好, 一直处于学术界边缘的地位。

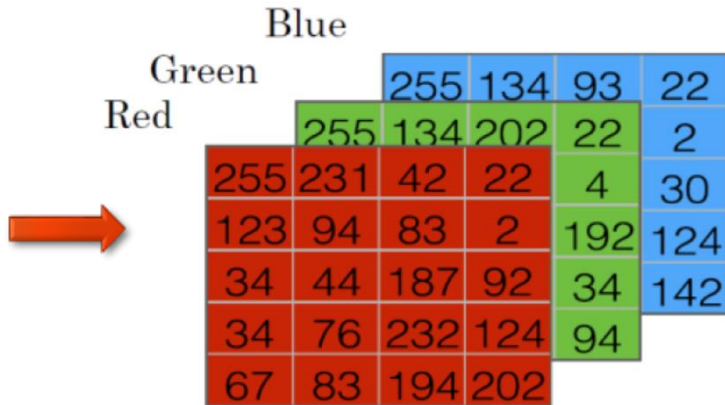
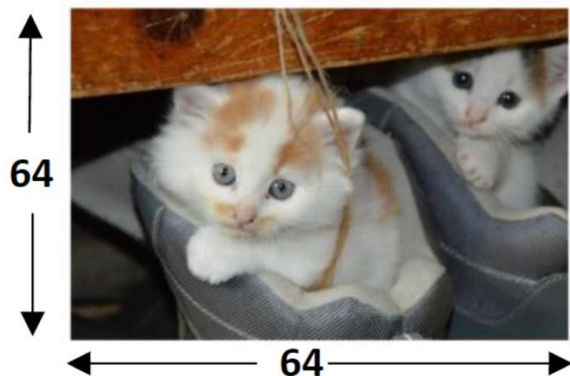
<https://www.aminer.cn/archive/gradient-based-learning-applied-to-document-recognition/53e9b85bb7602d970441f6c2>

- Hinton团队的论文ImageNet Classification with Deep Convolutional Neural Networks中提到的Alexnet引入了全新的深层结构和dropout方法，error rate从25%以上降到了15%，颠覆了图像识别领域。指出只要通过一些方法加深网络到8层左右，可大幅提升网络表达能力，取得非常好的效果
- LeCun组2013年提出一个Dropconnect，把error rate提升到了11%。
- 2015年以后，ResNet, Residual-Attention, DenseNet, SENet等也各有贡献，各自引入了Group convolution、Attention、Dense connection、channelwise-attention等，最终ImageNet上error rate降到了2.2%，完爆人类

论文地址：<https://www.aminer.cn/archive/imagenet-classification-with-deep-convolutional-neural-networks/53e9a281b7602d9702b88a98>

- 深度学习基本概述
- 卷积神经网络原理
 - 两种理解
 - 术语原理
- 常见的卷积神经网络结构
- 目标检测与分割

图像的数字表示

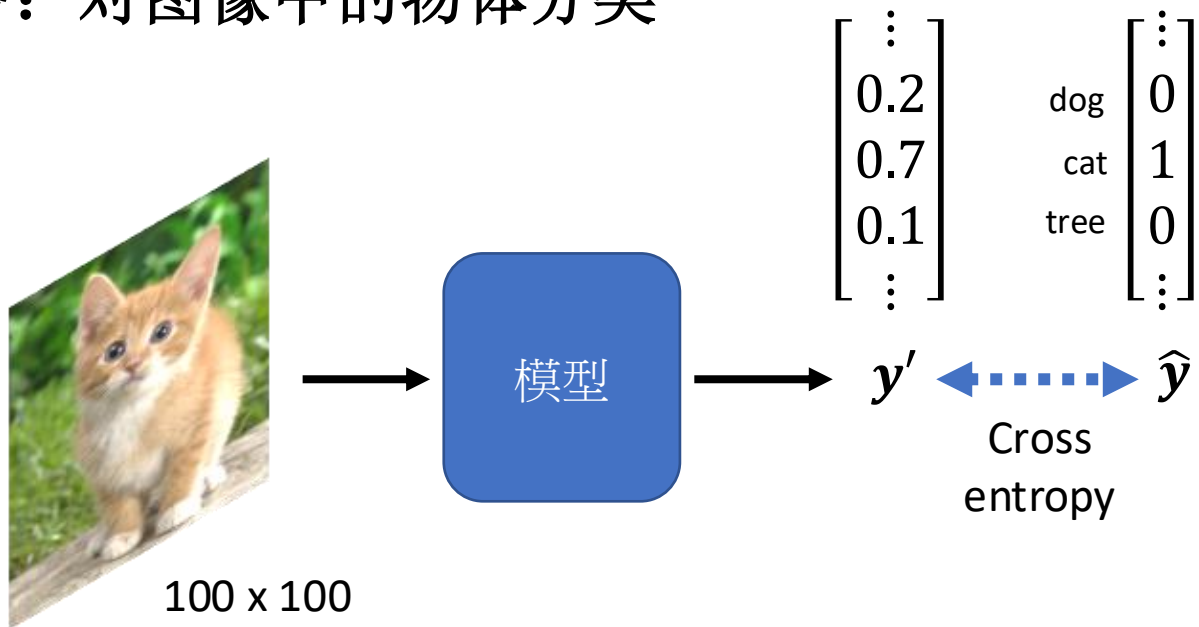


$$x = \begin{bmatrix} 255 \\ 231 \\ 42 \\ \vdots \\ 255 \\ 134 \\ 202 \\ \vdots \\ 255 \\ 134 \\ 93 \\ \vdots \end{bmatrix} \begin{matrix} \text{red} \\ \\ \\ \\ \text{green} \\ \\ \\ \text{blue} \end{matrix}$$

一张图片数据量是 $64 \times 64 \times 3$ ，因为每张图片都有3个颜色通道。
如果计算一下的话，可得知数据量为12288

图像分类示例1---以神经元角度理解

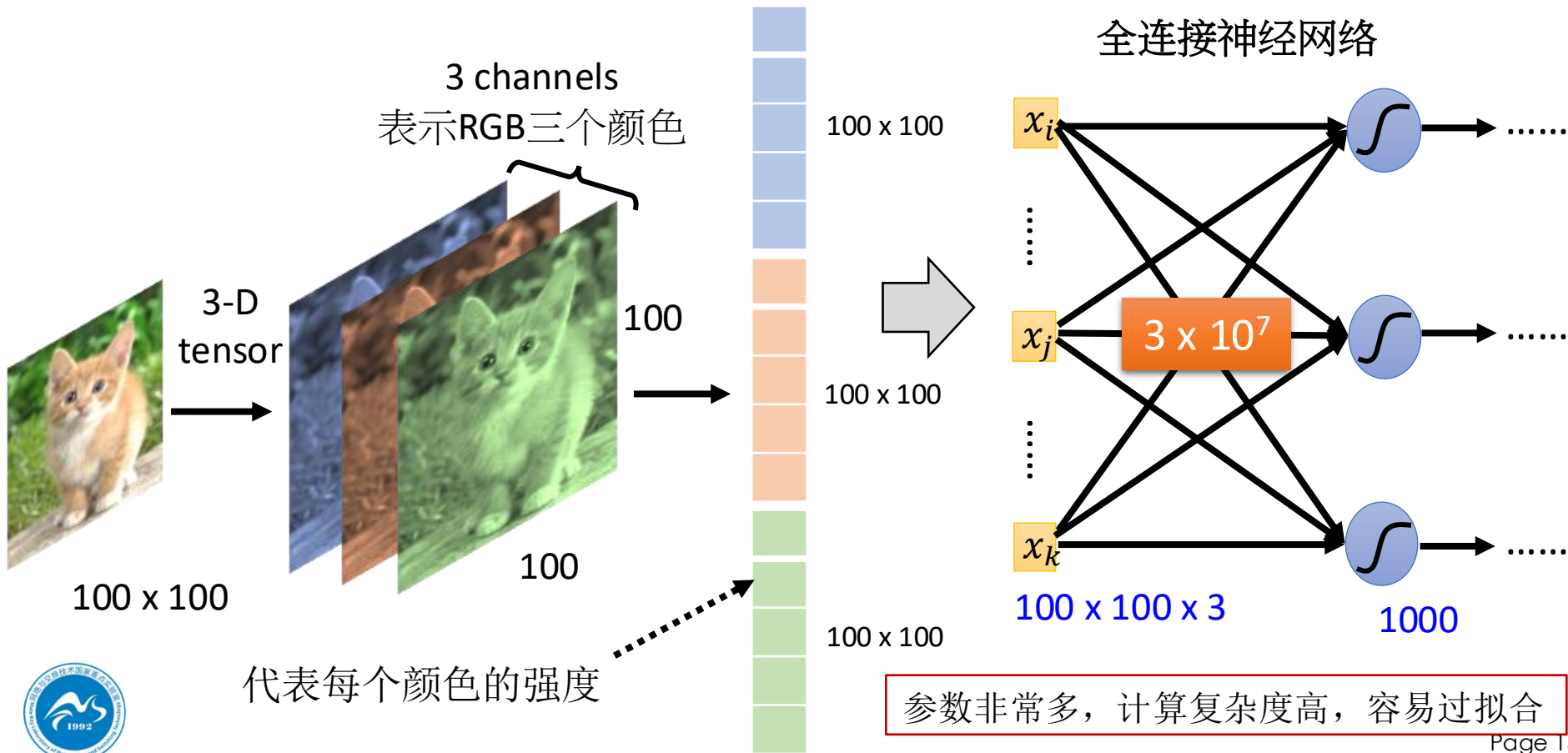
任务：对图像中的物体分类



图像已经预处理为同样大小

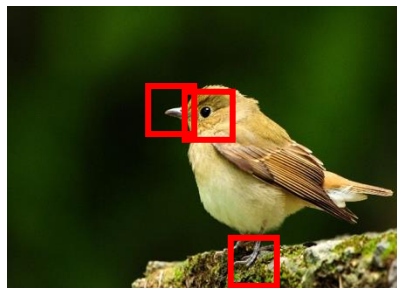
参考李宏毅老师《机器学习》

图像分类示例1

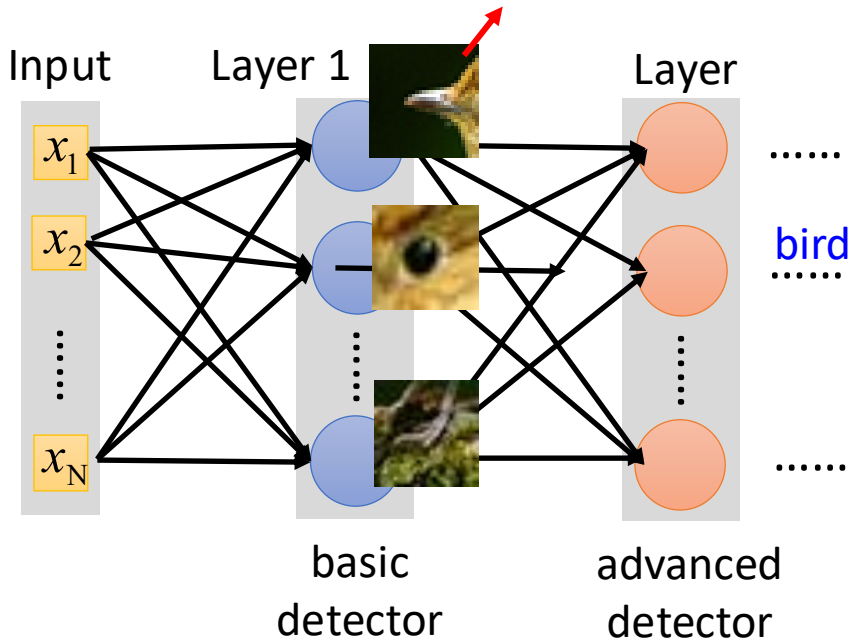


第一个观察

要识别鸟，是否需要看到全部图像？

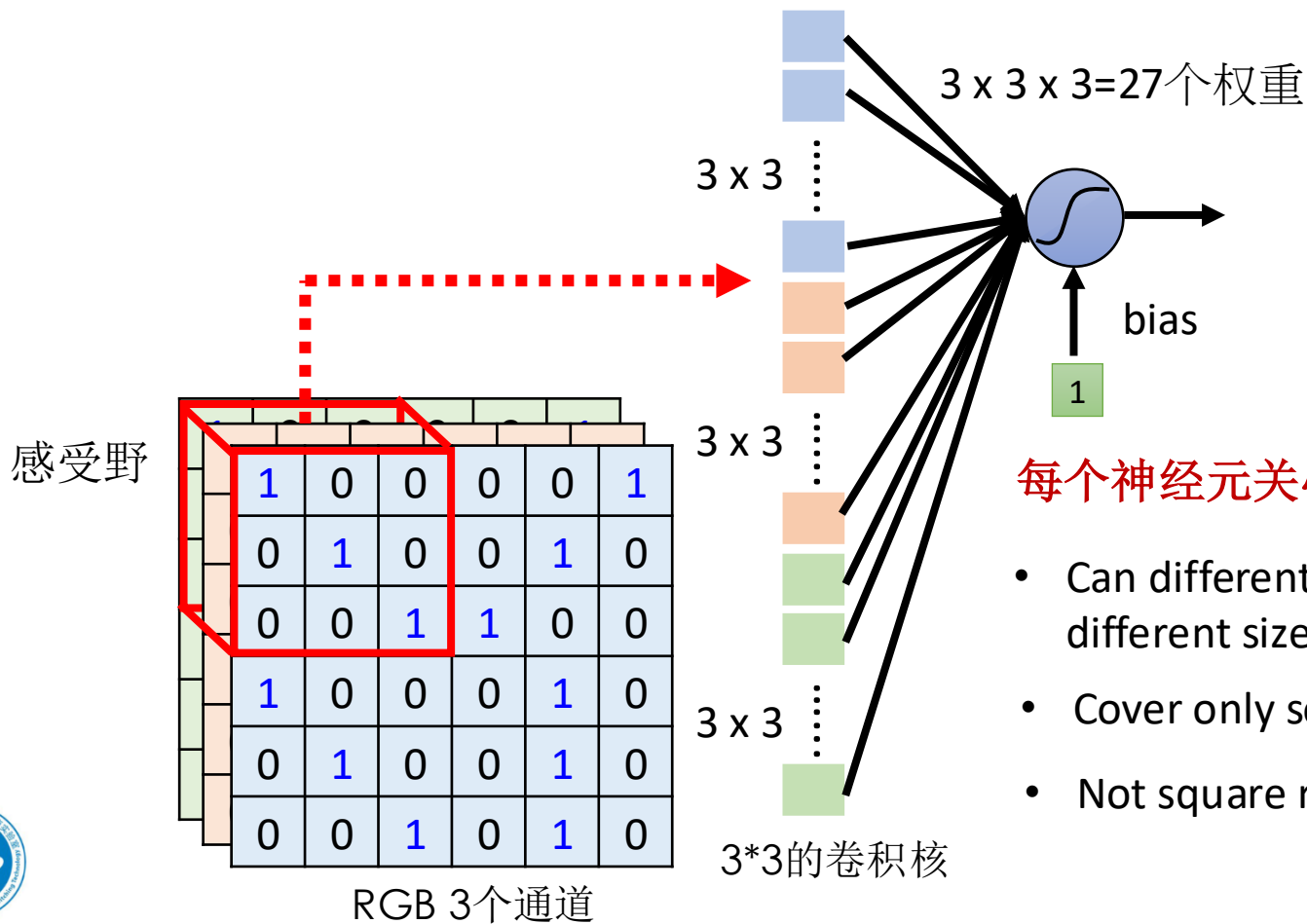


一个神经元不需要看到整个图像，识别关键特征即可（负责一小部分的特征）



Some patterns are much smaller than the whole image.

神经元的感受野与权重

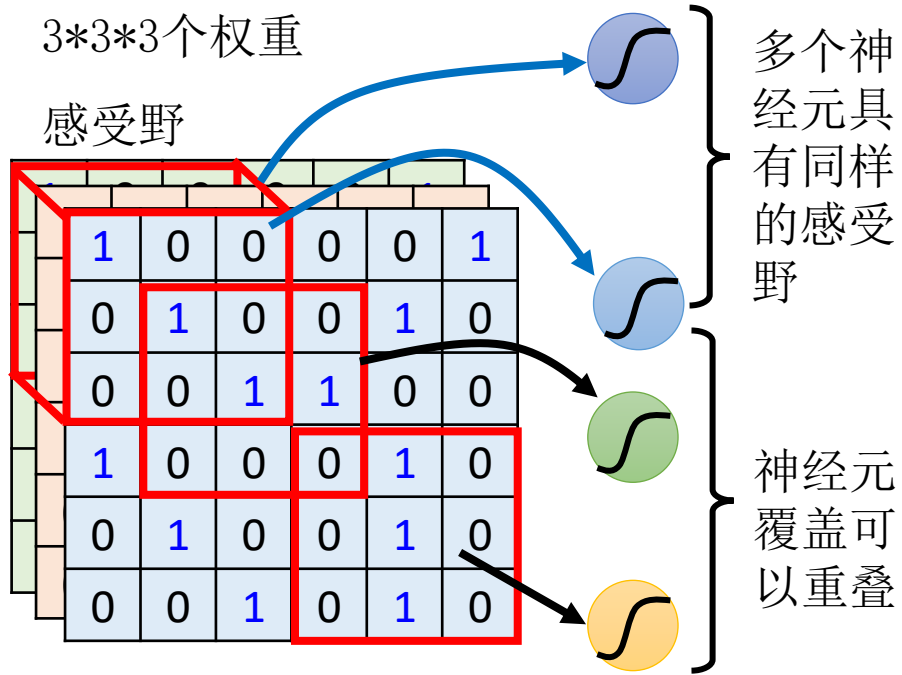


每个神经元关心自己负责的感受野

- Can different neurons have different sizes of receptive field?
- Cover only some channels?
- Not square receptive field?



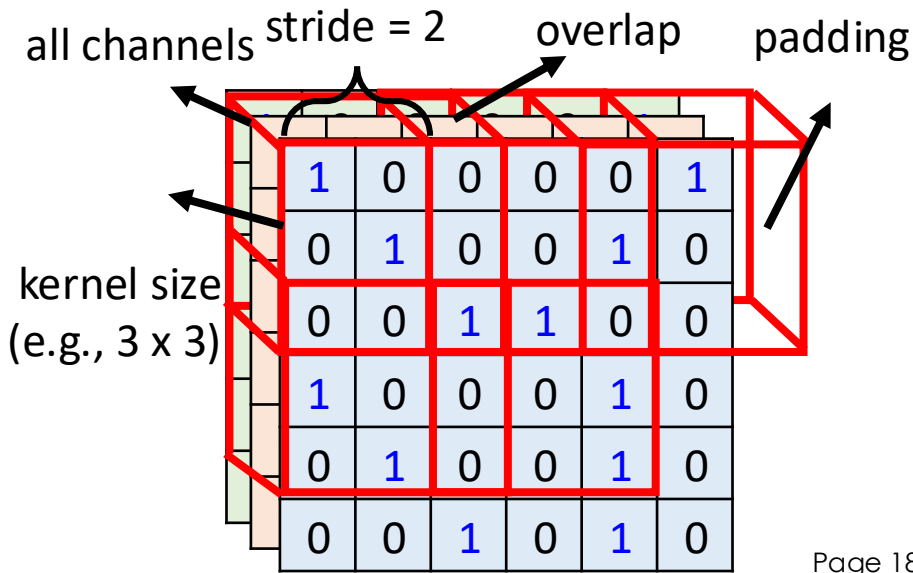
神经元的感受野与权重



经典的设计：感受野覆盖整个图像

每个感受野有一组神经元（e. g.，64个）

感受野之间高度重叠（有些特征正好出现在两个感受野交接的范围内）

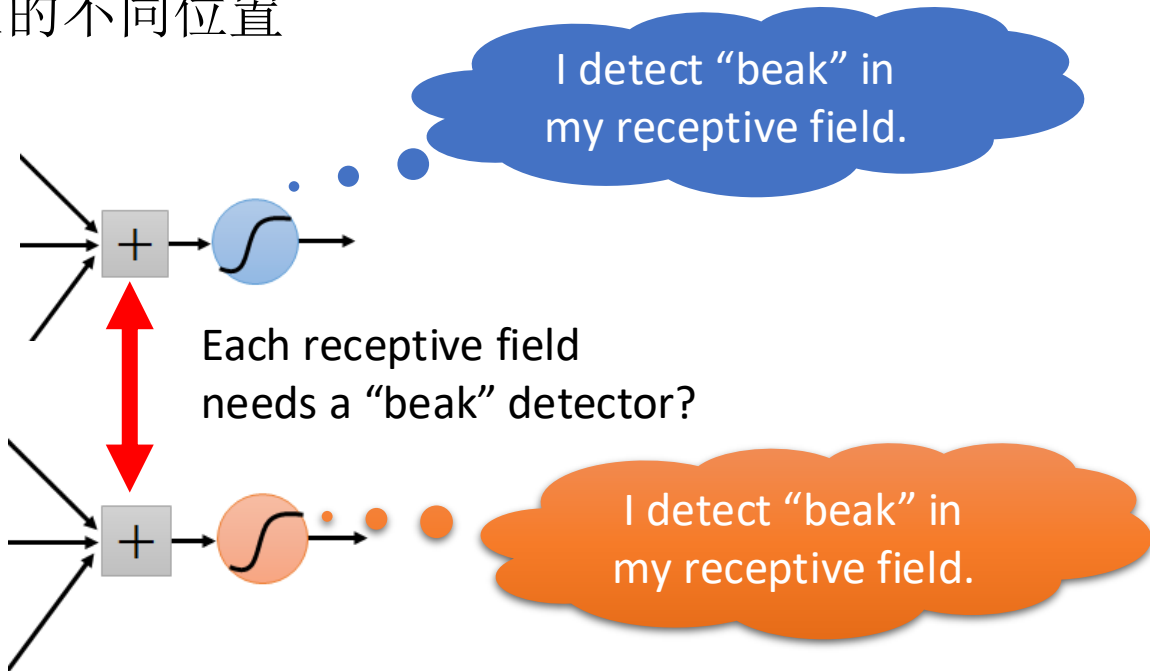




特征在不同位置

第二个观察

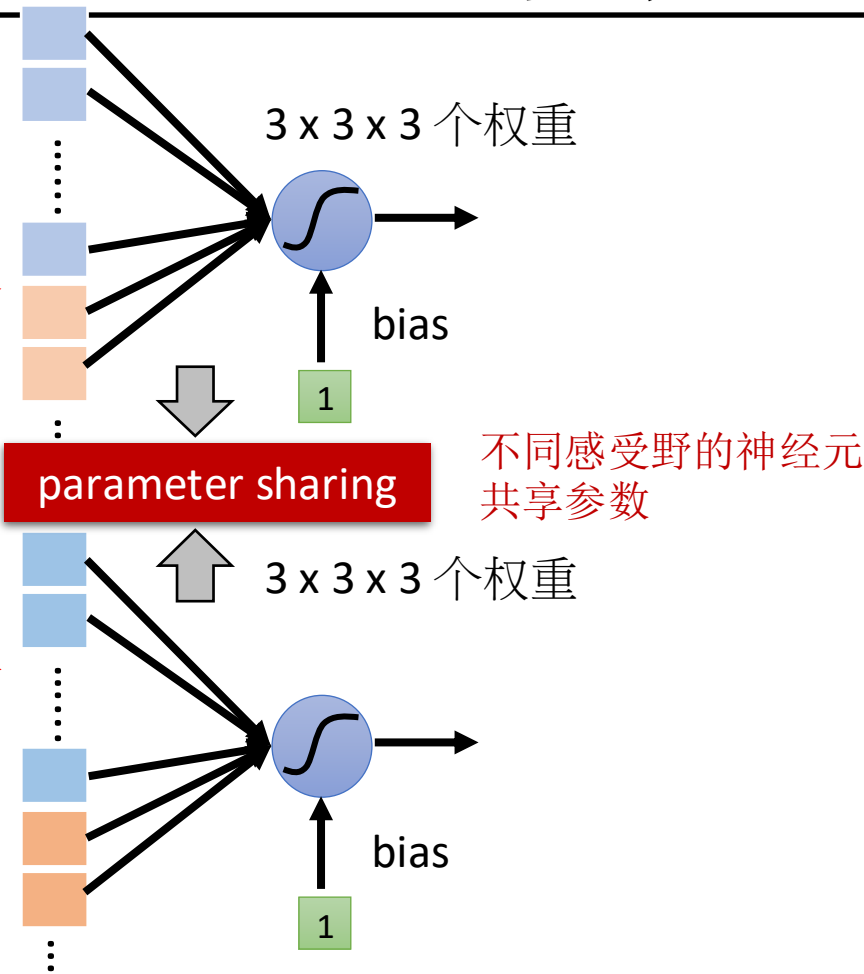
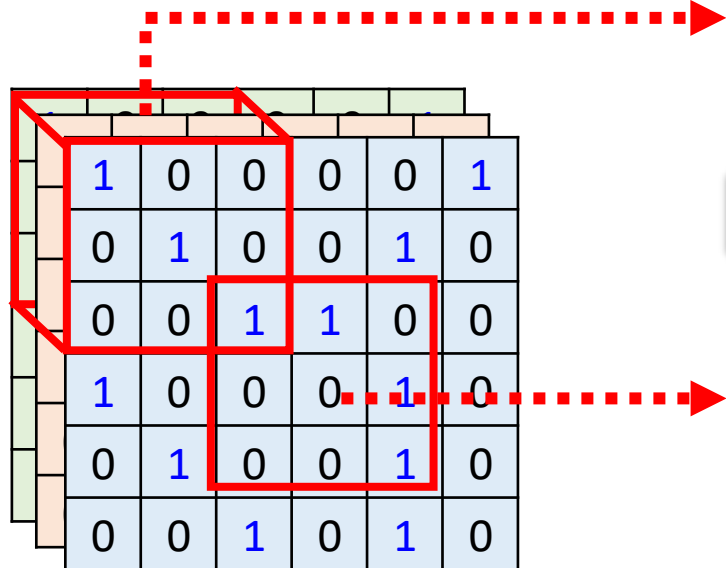
同样的特征出现在图像的不同位置



这些检测鸟嘴的神经元，识别的模式一样，只是针对鸟嘴的区域不同

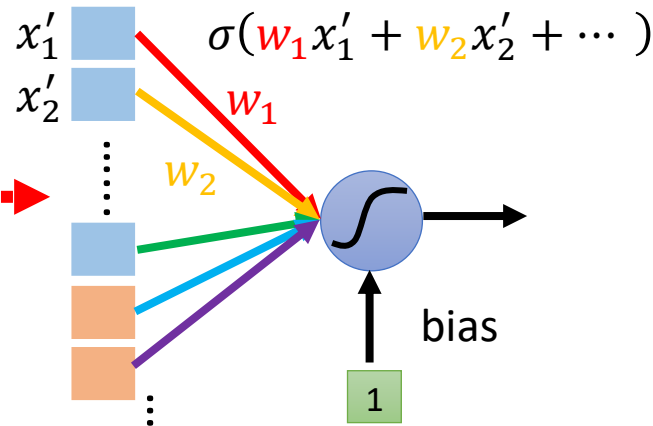
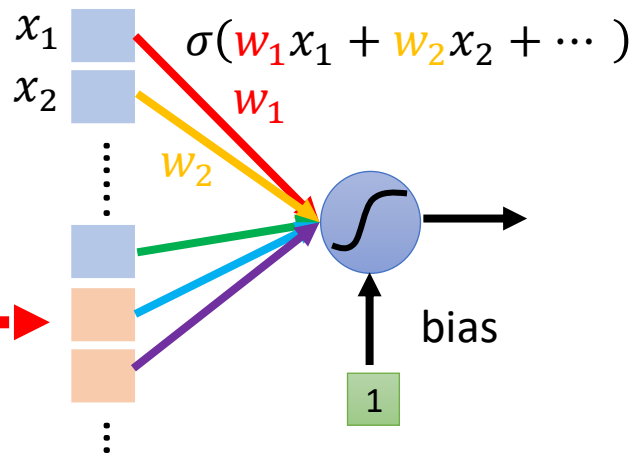
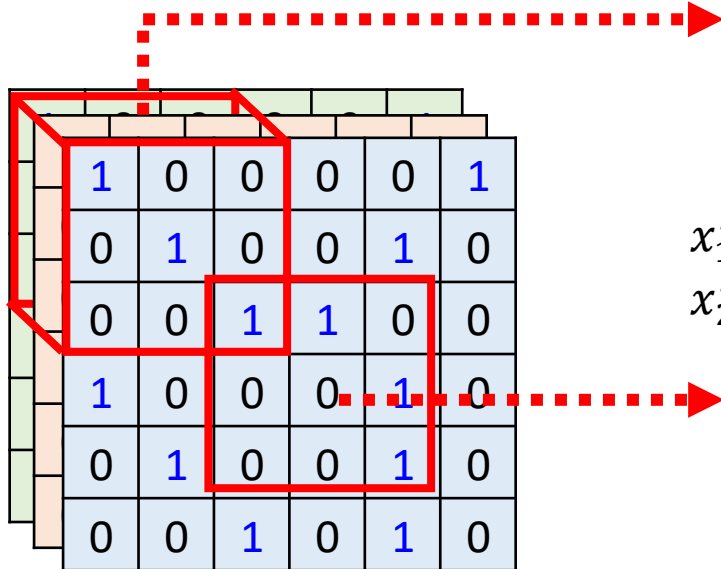


参数共享



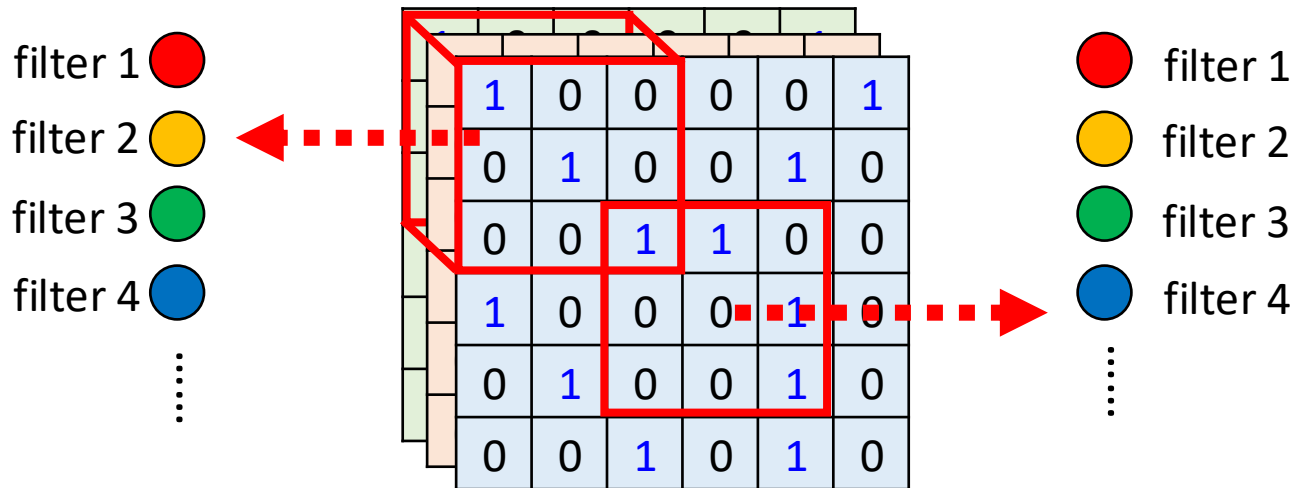
共享参数

两个感受野不同的神经元权重完全相同，
可以共享参数；相同感受野的神经元不能共享参数



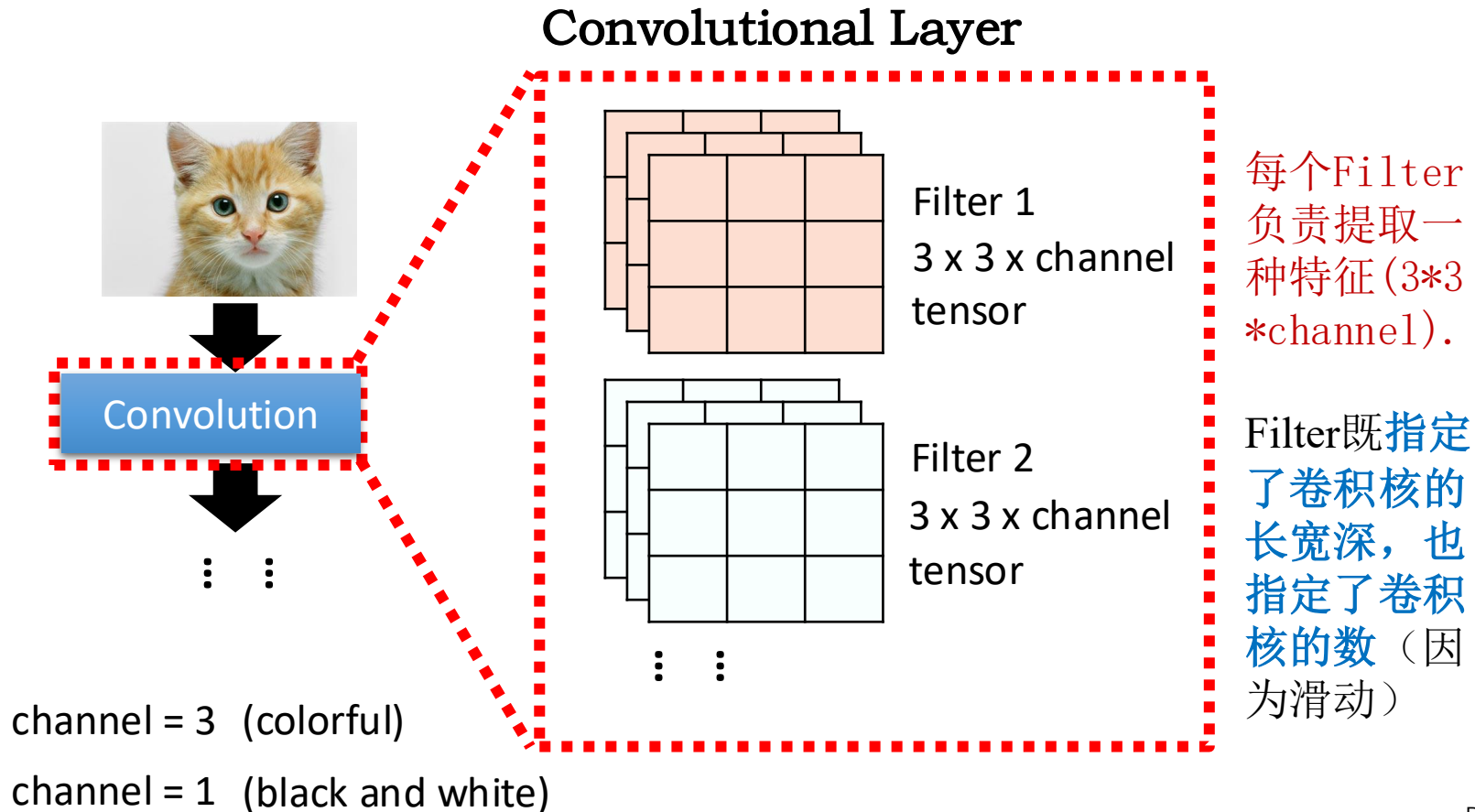
每个感受野都有一组神经元，例如64个

每个感受野有一组共用的参数，称为过滤器Filter





图像分类示例2---以Filter角度理解



1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Consider channel = 1 (black and white image)

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

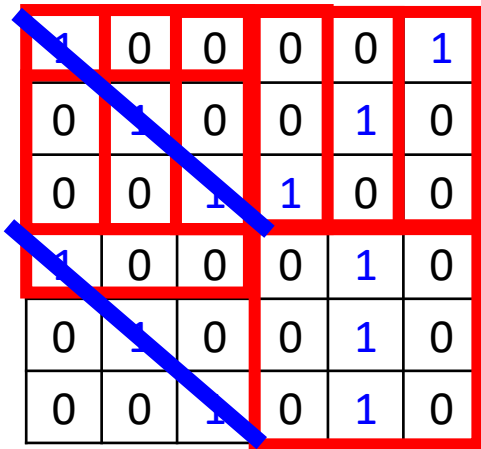
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

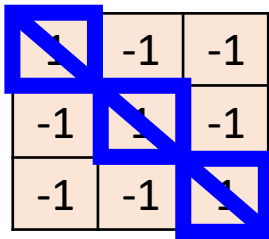
⋮ ⋮

实际上这些filter里的数值依靠梯度下降计算的

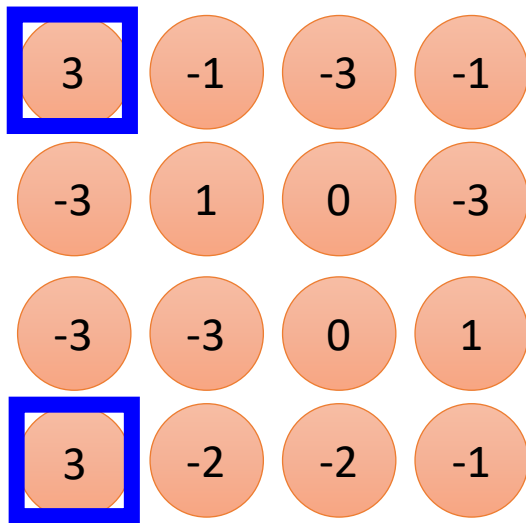
stride=1



6 x 6 image



Filter 1



图像中有
连着3个1,
filter数值
最大

stride=1

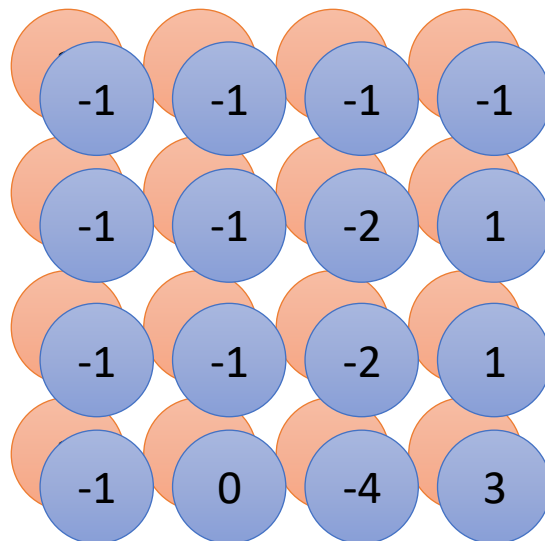
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Do the same process for every filter

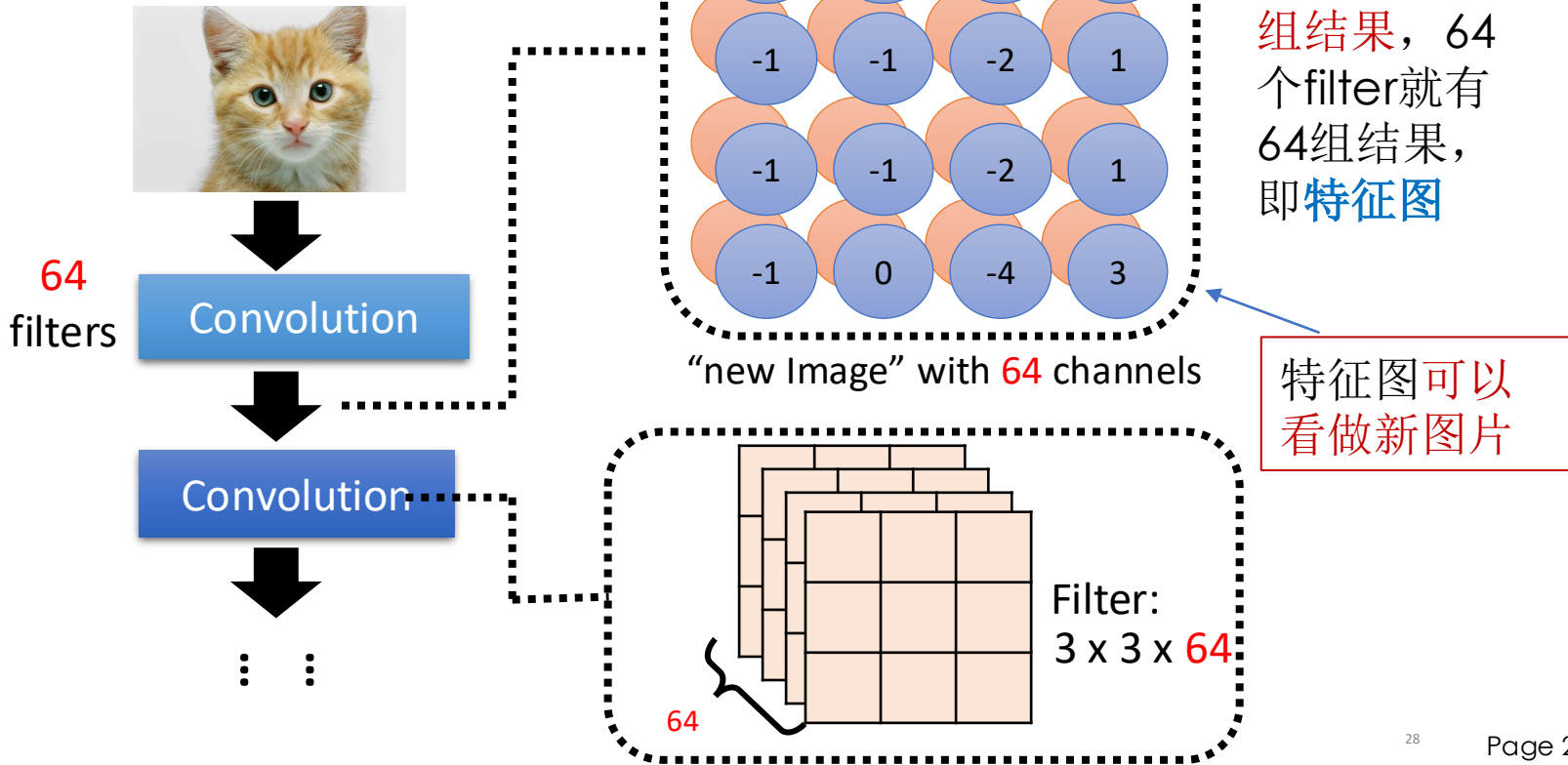


特征图
Feature
Map

4 x 4 image



多个卷积层叠加



64
filters

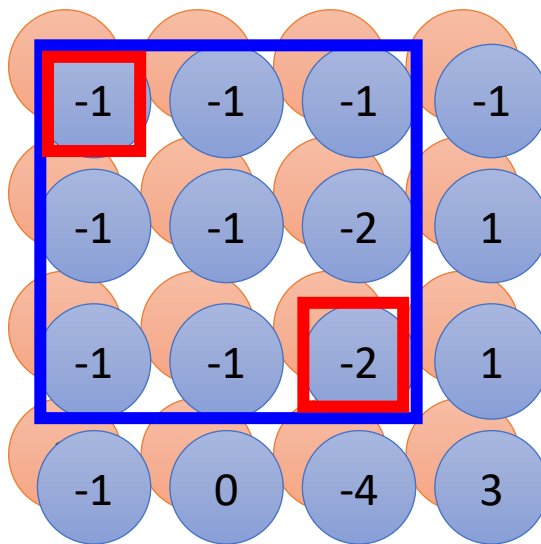


Convolution

Convolution

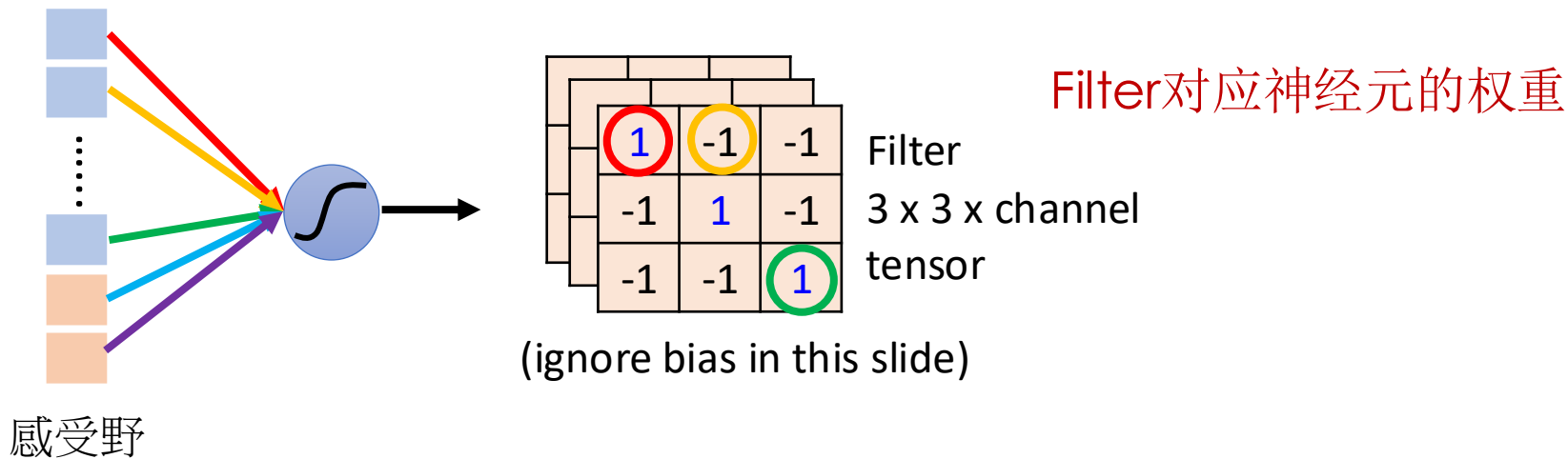
⋮ ⋮

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



通过叠加卷积层，可以让卷积核看到的范围越来越大

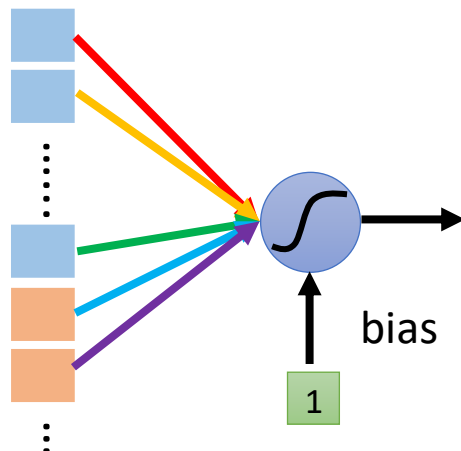
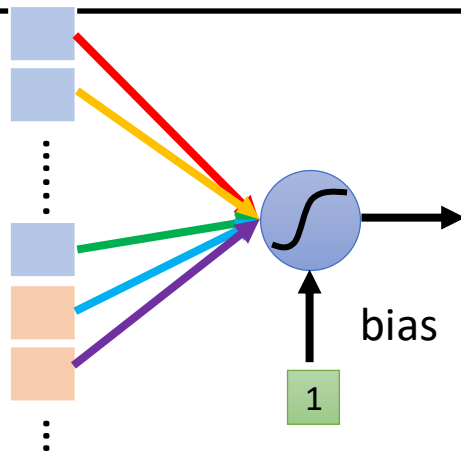
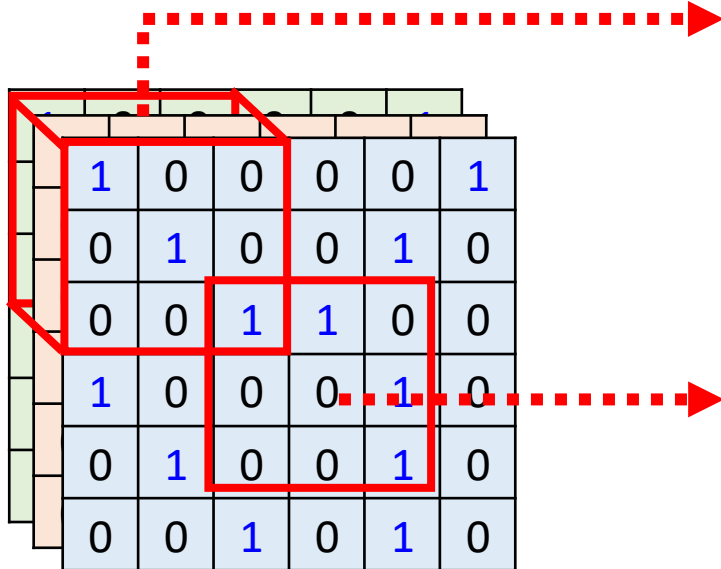
比较从神经元角度理解与从Filter角度理解





比较两种理解方式

具有不同感受野的神经元共享权重，
相当于**每个过滤器都扫过图像**





■ 第三个观察：对像素下采样不会改变物体



一张大图做下采样

池化 Pooling – Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

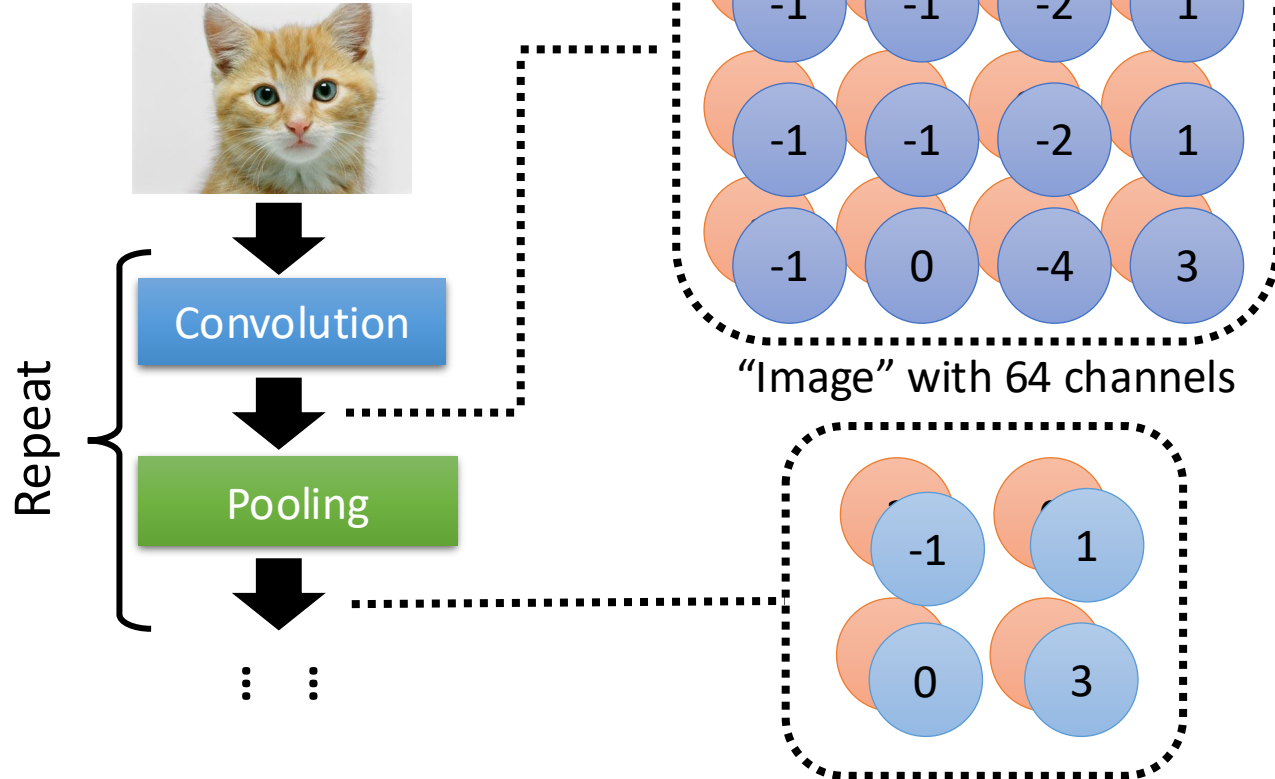
-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

没有参数
不需要学

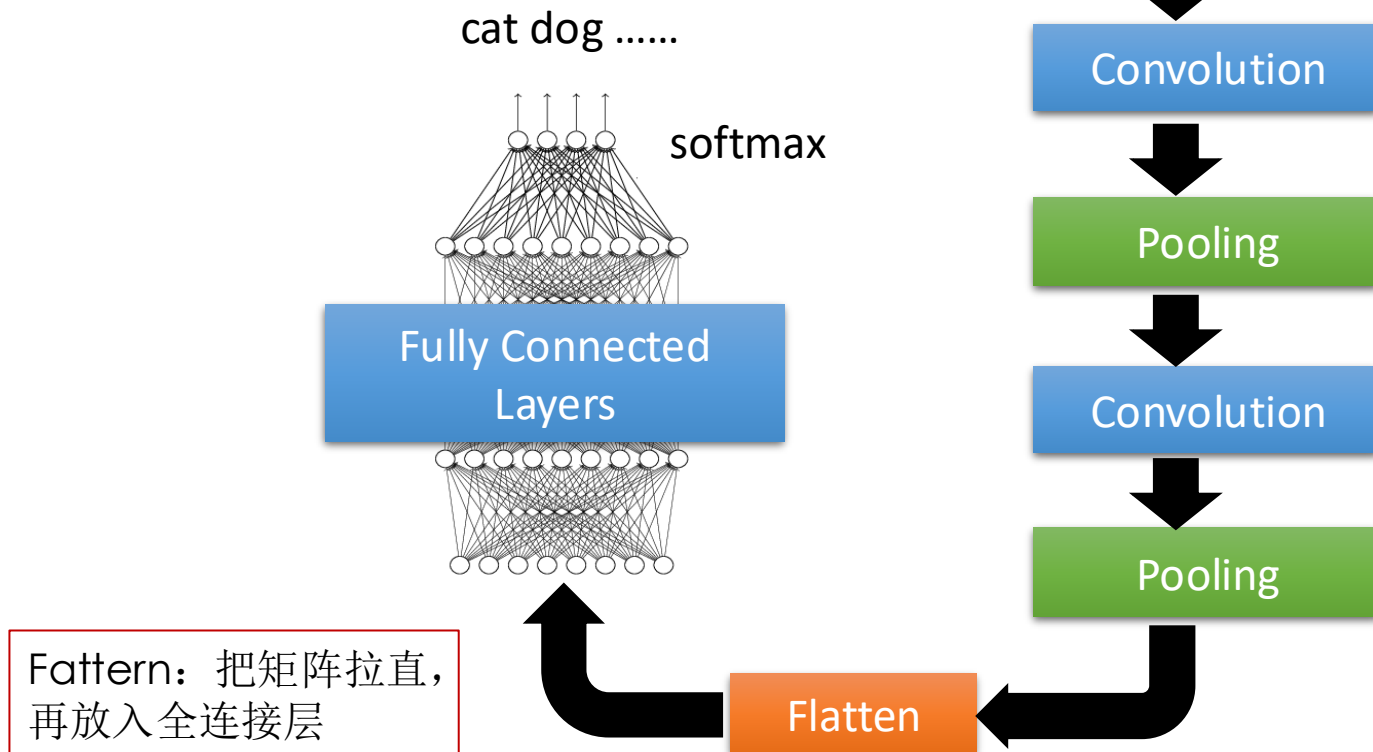
2*2个一组，
选最大的代表

池化减少运算量，
但对性能有影响

Convolutional Layers+ Pooling



一个典型的CNN



卷积核是一种变换

例如，傅里叶变换 $F(j\omega) = \int_{-\infty}^{+\infty} f(t)e^{-j\omega t} dt$

图像提取特征之前用的比较多的是卷积核是Gabor核，
提取纹理特征

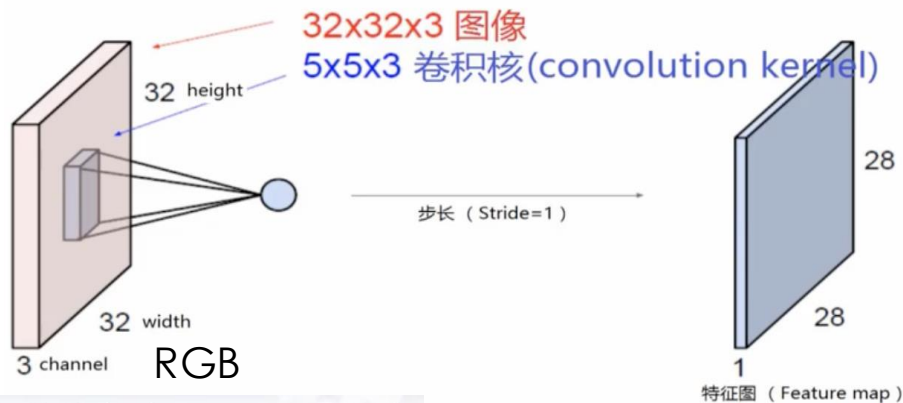
之前人为设计，现在是自动学习卷积核

将原来由手工设计卷积核变成自动学习卷积核



- 卷积核**Kernel**
- 通道**Channel**
- 步长**Stride**
- **Feature Map**
- 补零**Padding**

以LeNet为例



可以将图像卷积看成全连接网络的权值共享 (weight sharing)

x1	x2	x3
x4	x5	x6
x7	x8	x9

 $\star$

w1	w2
w3	w4

 $+$

b1	b2
b3	b4

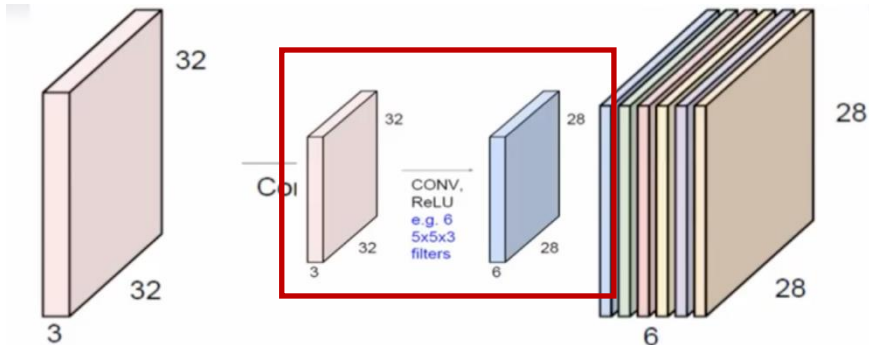
 $=$

p1	p2
p3	p4

$$\begin{aligned}
 p1 &= w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x4 + w4 \cdot x5 + b1 \\
 p2 &= w1 \cdot x2 + w2 \cdot x3 + w3 \cdot x5 + w4 \cdot x6 + b2 \\
 p3 &= w1 \cdot x4 + w2 \cdot x5 + w3 \cdot x7 + w4 \cdot x9 + b3 \\
 p4 &= w1 \cdot x5 + w2 \cdot x6 + w3 \cdot x8 + w4 \cdot x9 + b4
 \end{aligned}$$

图像大小: M, N
 卷积核大小: m, m
 步长: μ, ν
 特征图大小: K, L



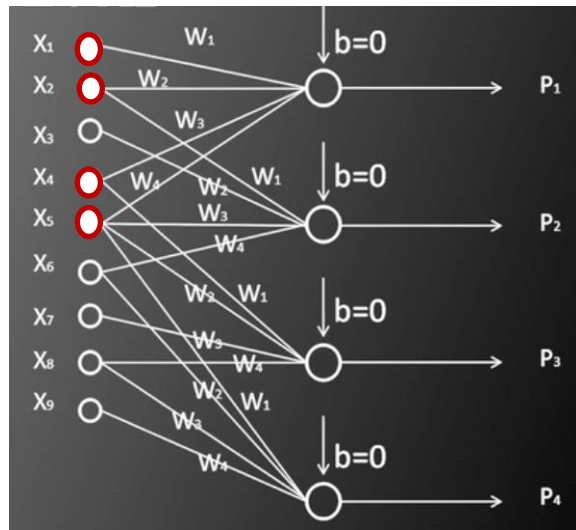


如果我们用6个卷积核，就能获得6个特征图（feature map）。你也可以把产生的6个特征图看成一个新的“图像”，其height, width, channel数目分别是28, 28, 6。

■ 非线性层ReLU：卷积后经过ReLU函数变换

如果采用5*5*3卷积核，再加上自带的偏置，共有参数 $(5*5*3+1)*6=456$ 个

■ 共享权重



w1被4个神经元共享

w2被4个神经元共享

w3被4个神经元共享

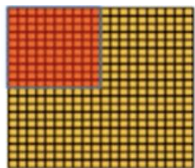
w4被4个神经元共享

相当于全连接神经网络中一些w为0

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial p_1} x_1 + \frac{\partial E}{\partial p_2} x_2 + \frac{\partial E}{\partial p_3} x_4 + \frac{\partial E}{\partial p_4} x_5$$

$$p_1 = w_1 * x_1 + w_2 * x_2 + w_3 * x_4 + w_4 * x_5 + b_1$$

- 下采样层：LeNet采用平均下采样average subsampling



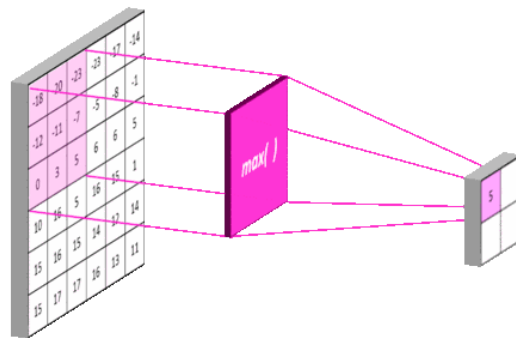
convolved feature

28*28特征图



subsamped feature

14*14特征图

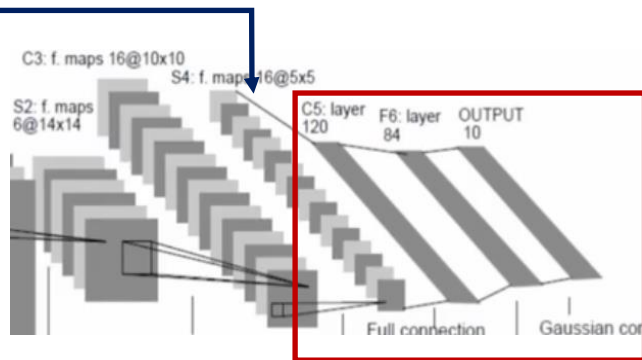


- 下采样层的后向传播

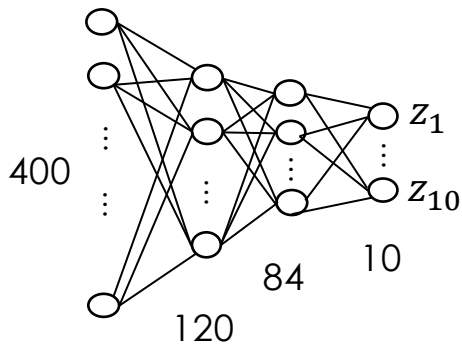
对相邻四个像素 x_1, x_2, x_3, x_4 取平均值，获得 $y = \sum_{i=1}^n x_i$ ，

$$\text{则有 } \frac{\partial E}{\partial x_1} = \frac{\partial E}{\partial x_2} = \frac{\partial E}{\partial x_3} = \frac{\partial E}{\partial x_4} = \frac{1}{4} y.$$

用16个5*5*6的卷积核，stride=1
作用于14*14*6的特征图



■ 全连接层：特征图完全展开



计算速度取决于卷积层，
空间取决于全连接层

$$\text{Softmax}(z)=q \quad E = - \sum_{i=1}^{10} Y_i \log(q_i)$$

主要参数在全连接层，LeNet参数个数计算：

参数个数计算：

第1层(convolutional layer): $(5*5+1)*6=156$

第2层(subsampling layer): 0

第3层(convolutional layer): $(5*5*6+1)*16=2416$

第4层(subsampling layer): 0

第5层(fully connected layer): $(5*5*16+1)*120=48120$

第6层(fully connected layer): $(120+1)*84=10164$

第7层(fully connected layer): $(84+1)*10=850$

参数总数: 61,706

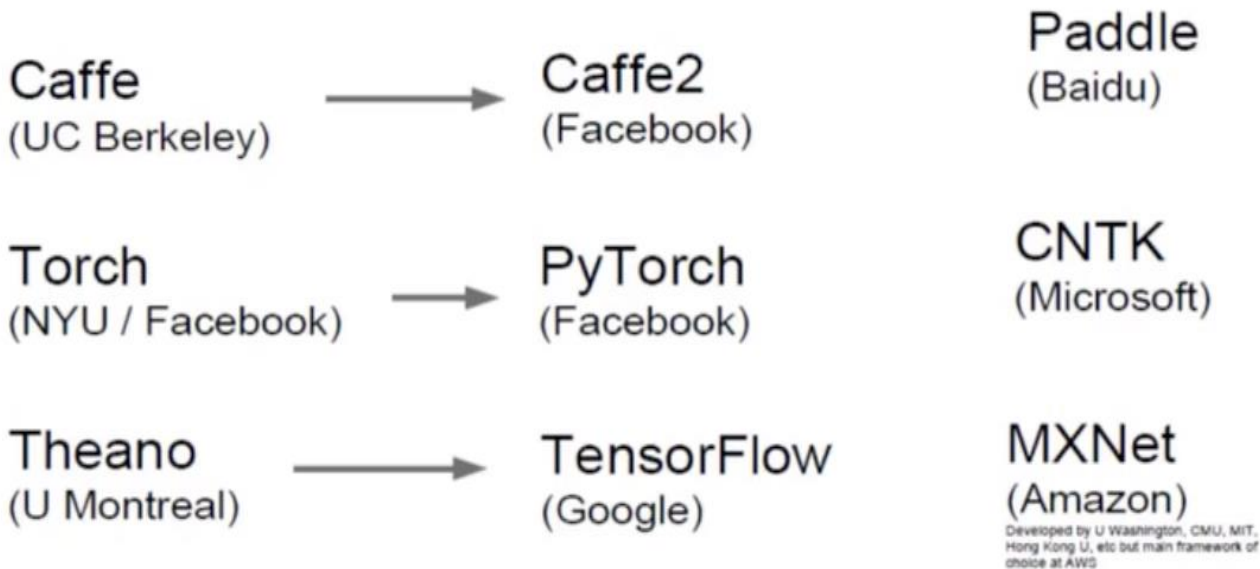


- 手写数字的MNIST数据集（LeCun 1998年）
- 60,000个用于训练的示例和10,000个用于测试的示例
- 图像是固定大小(28x28像素)
- 每个图像都被平展并转换为784($28 * 28$)个特征的一维数组



- ImageNet: Fei-fei Li, 2007年
 - 1500万张图片，1TB左右的数据，对应了2万多类
- 常用的是ISLVR 2012 (*ImageNet Large Scale Visual Recognition Challenge*) 比赛用的子数据集
 - 训练集: 1,281,167张图片+标签
 - 验证集: 50,000张图片+标签
 - 测试集: 100,000张图片
 - 一个文件对应着一个类别，一个一千类，每个类有一千多张图

<http://www.image-net.org>



推荐B站视频计算所陈云霄老师《智能计算系统》



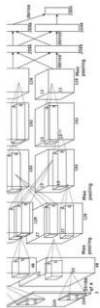
- 深度学习基本概述
- 卷积神经网络原理
- 常见的卷积神经网络结构
 - AlexNet
 - GoogleNet
 - ResNet
 - DensNet
- 目标检测与分割



常见的深度神经网络

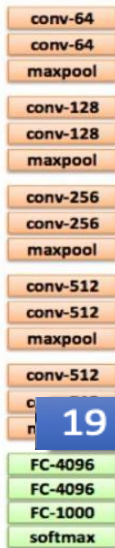
8 layers

16.4%



AlexNet (2012)

7.3%

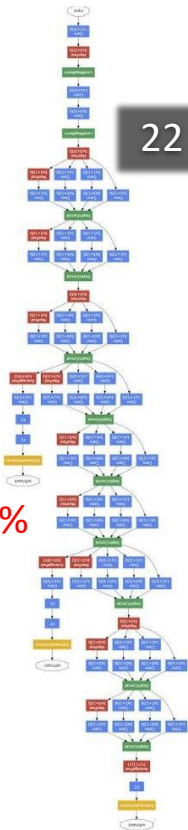


19 layers

VGG (2014)

22 layers

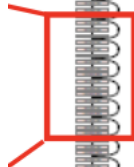
6.7%



GoogleNet (2014)

152 layers

3.57%



Residual Net
(2015)

参数超过6000万的大规模神经网络

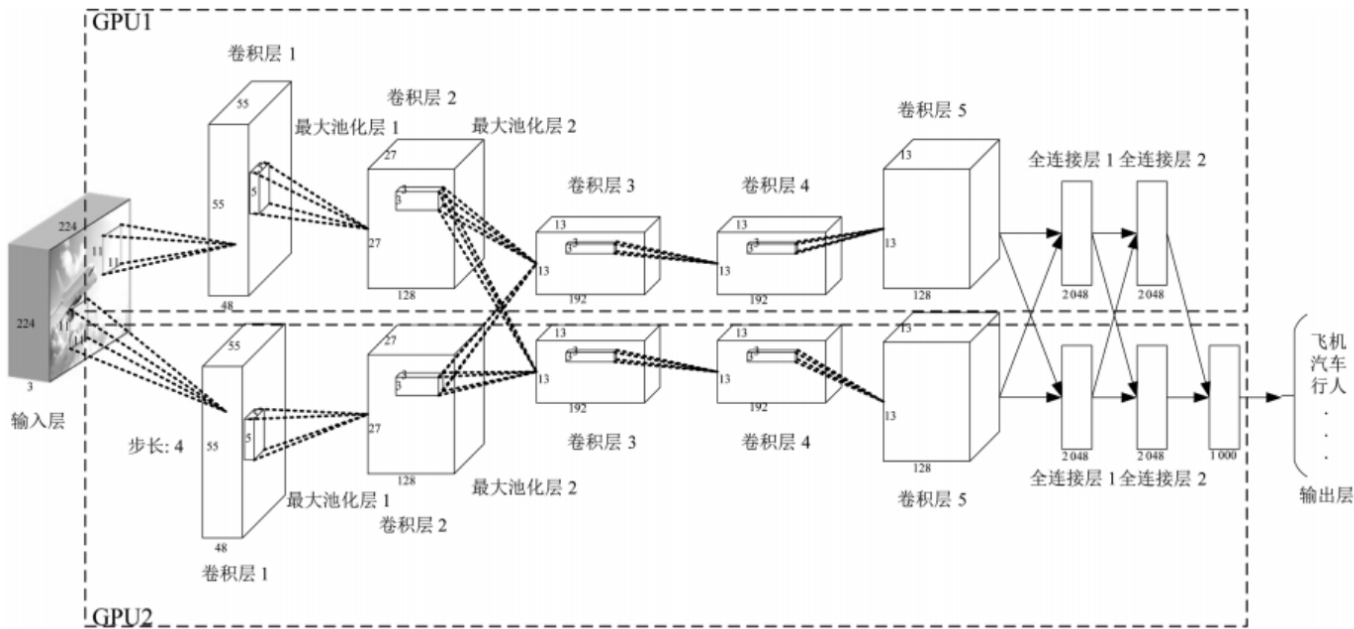


图2 AlexNet 卷积神经网络结构示意图^[8]

AlexNet包含了6亿三千万个连接，6000万个参数和65万个神经元，拥有**5个卷积层**，其中3个卷积层后面连接了最大池化层，最后还有3个全连接层。

并行结构

A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing 25, MIT Press, Cambridge, MA, 2012.



AlexNet结构



```
1 class AlexNet(nn.Module):
2     def __init__(self,num_classes=2):
3         super(AlexNet, self).__init__()
4         self.features=nn.Sequential(
5             nn.Conv2d(3,48, kernel_size=11),
6             nn.ReLU(inplace=True),
7             nn.MaxPool2d(kernel_size=3,stride=2),
8             nn.Conv2d(48,128, kernel_size=5, padding=2),
9             nn.ReLU(inplace=True),
10            nn.MaxPool2d(kernel_size=3,stride=2),
11            nn.Conv2d(128,192,kernel_size=3,stride=1,padding=1),
12            nn.ReLU(inplace=True),
13            nn.Conv2d(192,192,kernel_size=3,stride=1,padding=1),
14            nn.ReLU(inplace=True),
15            nn.Conv2d(192,128,kernel_size=3,stride=1,padding=1),
16            nn.ReLU(inplace=True),
17            nn.MaxPool2d(kernel_size=3,stride=2),
18        )
```

卷积层

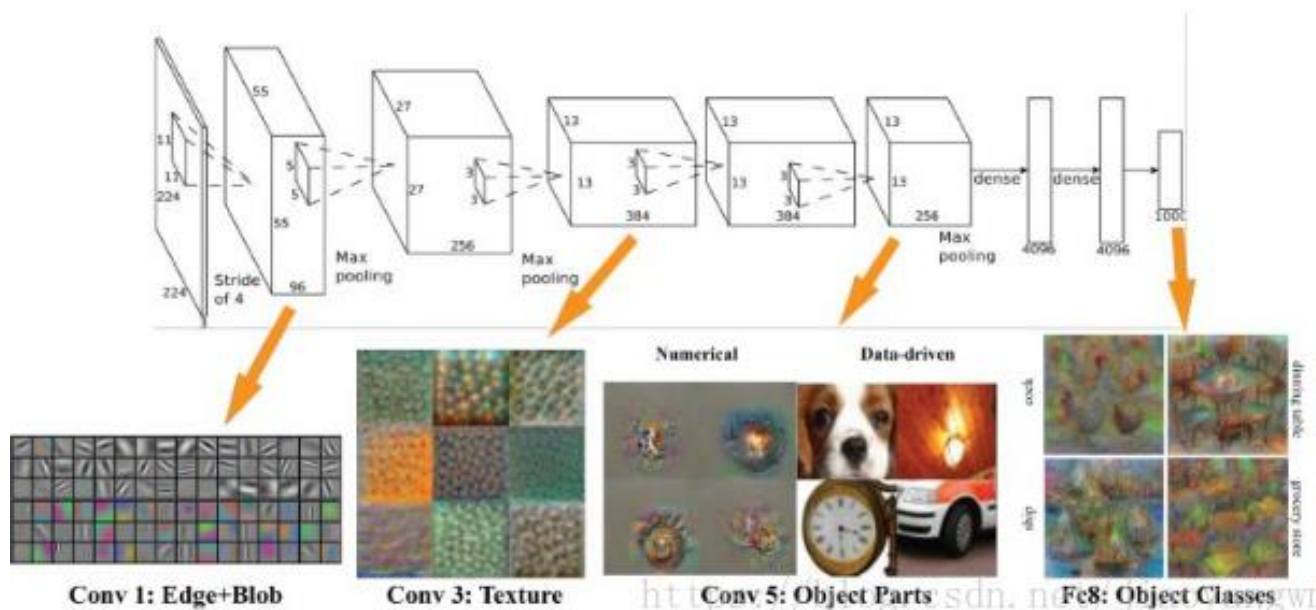
激活函数

池化层



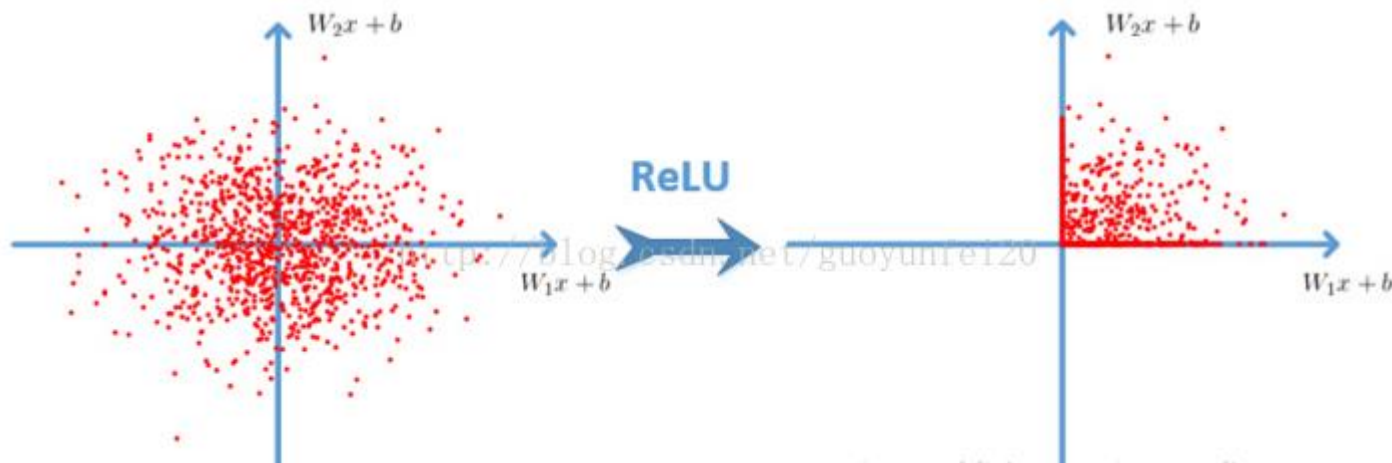
AlexNet层次

- 第一层：一些填充的块状物和边界等特征
- 中间层：学习一些纹理特征
- 更高层：接近于分类器的层级，可以明显的看到物体的形状特征
- 最后一层：分类层，完全是物体的不同的姿态，根据不同的物体展现出不同姿态的特征。





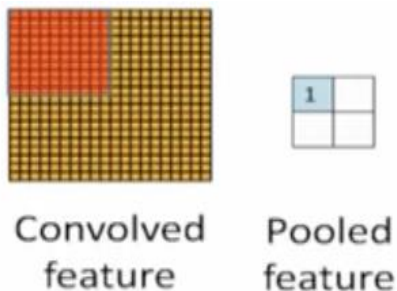
- Relu函数: $f(x)=\max(0,x)$
- 采用非饱和线性单元——ReLU代替传统的经常使用的tanh和sigmoid函数，**加速了网络训练的速度**，降低了计算的复杂度，对各种干扰更加具有鲁棒性，并且在一定程度上避免了梯度消失问题。
- 实践证明，改为Relu，可使网络训练以更快速度收敛





Alex关键改进---最大池化

- 池化：将邻近像素作为一个“池子”重新考虑



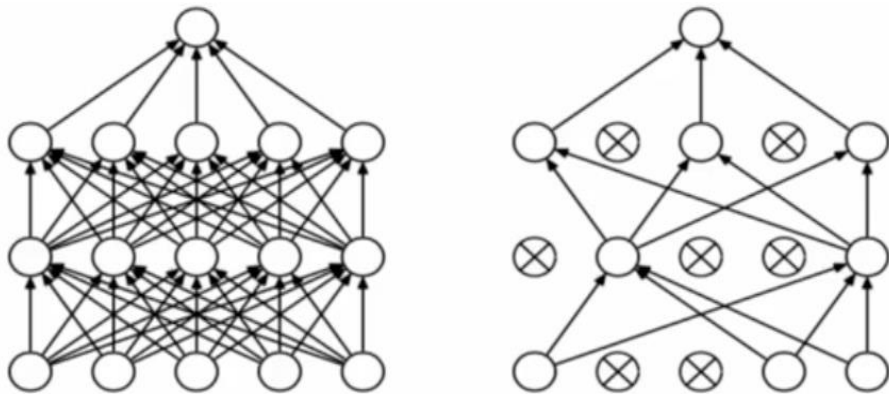
- AlexNet中，**提出最大池化（Max Pooling）概念**，即对每一个邻近像素组成的池子，选取像素最大值作为输出，**避免了平均池化所带来的模糊化的效果**。
- LeNet中池化的像素不重叠，而AlexNet进行有重叠的池化（步长<池化核）。实践表明，**有重叠的最大池化**，提升了特征的丰富性，可以很好的克服过拟合问题，提升性能。





Alex关键改进---随机丢弃

- **随机丢弃 (Dropout)**：为了避免模型参数更新过快导致过拟合，每次利用训练样本更新参数时，随机丢弃一定比例的神经元，被丢弃的神经元将不参加训练过程，输入和输出该神经元的权重系数也不做更新。这样每次训练时，训练网络架构都不一样，而这些不同的网络架构却分享相同的权重系数。实验表明，随机丢弃技术减缓了网络收敛速度，也可以大概率避免过拟合。



Dropout做法是，对每一层，每次训练时以概率 p 丢弃一些神经元，这样每次训练的网络都不一样。

训练结束后的测试流程，要用完整的网络结构，同时对该层的所有参数 (W, b) 都要乘以 $(1-p)$ 。

Alex关键改进---增加样本&GPU训练

■ 增加训练样本。尽管ImageNet的训练样本数量有超过120万幅图片，但相对于6亿待估计参数来说，训练图像仍然不够。**Alex**等人采用了多种方法增加训练样本，包括：1. 将原图水平翻转；2. 将 256×256 的图像随机选取 224×224 的片段作为输入图像。运用上面两种方法的组合可以将一幅图像变为2048幅图像。还可以对每幅图片引入一定的噪声，构成新的图像。这样做可以较大规模增加训练样本，避免由于训练样本不够造成的性能损失

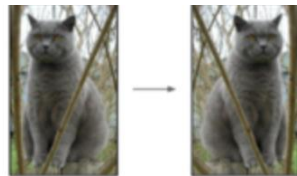
■ 用GPU加速训练过程。采用2片GTX 580 GPU对训练过程进行加速，由于GPU强大的并行计算能力，使得训练过程的时间缩短数十倍，哪怕这样，训练时间仍然用了六天。

Spot the GPUs!
(graphics processing unit)

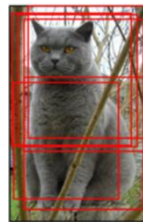


只有3G显存

水平翻转



随机裁剪、平移旋转



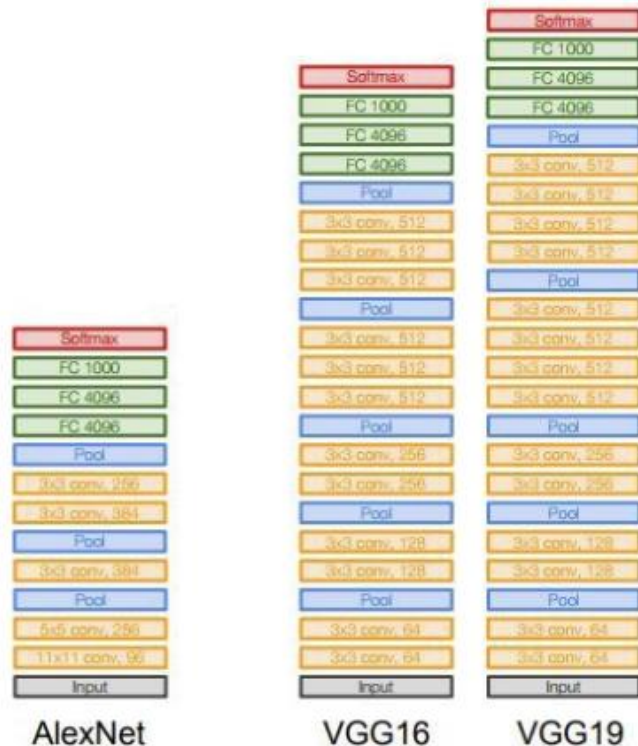
颜色变换



Alex Krizhevsky等人在ImageNet的测试集上获得了37.5%的Top 1错误率（即正确的类不是测试中排名第一的类的百分率）和16.4%的Top 5错误率（即正确的类不是测试中排名前五的类的百分率），这比第二名45.7%的Top 1错误率和25.8%的Top 5错误率高出了8个百分点。考虑到第二名与第三名的差距只有1个百分点左右，这个结果可以说是遥遥领先。

AlexNet火热，带动了深度学习！

- **VGGNet**: 牛津大学计算机视觉组（Visual Geometry Group）和Google DeepMind公司的研究员一起研发的深度卷积神经网络。
- 一系列结构包括**VGG-11**、**VGG-16**、**VGG-19**等，具有不同层数，参数也不同。
- VGGNet探索了卷积神经网络的**深度与其性能之间的关系**，通过反复堆叠3*3的小型卷积核和2*2的最大池化层，构筑了16~19层深的卷积神经网络。
- VGGNet相比之前最先进网络结构，错误率大幅下降，并取得了ILSVRC 2014比赛分类项目的第2名和定位项目的第1名。





ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

A网络（11层）有8个卷积层和3个全连接层；

E网络（19层）有16个卷积层和3个全连接层，卷积层宽度（通道数）从64到512，每经过一次池化操作，扩大一倍。

输入：训练时输入大小为224x224大小的RGB图像；

预处理：在训练集中的每个像素减去RGB的均值

卷积核：3x3大小的卷积核，或1x1的小卷积核，这种1x1卷积可看做是对输入通道的线性变换。

步长：步长stride为1

填充：填充1个像素

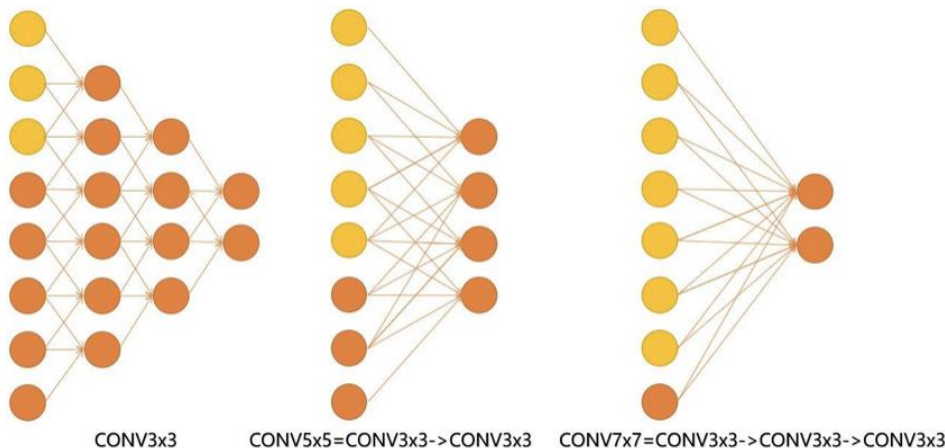
池化：max-pooling，共有5层在一部分卷积层之后，连接的max-pooling的窗口是2x2，步长为2

全连接层：前两个全连接层均有4096个通道，第三个全连接层由1000个通道，用来分类。所有网络的全连接层配置相同。

激活函数：ReLU

不使用LRN（局部响应）：这种归一化不能带来很大的提升，反而会导致更多的内存消耗和计算时间





input=8, 3层conv3x3后, output=2,
等同于1层conv7x7的结果;
input=8, 2层conv3x3后, output=2,
等同于2层conv5x5的结果。

3个3*3卷积层具有7x7的有效感受野。
每个最大池化层后滤波器的数量增加一倍, 进一步加强了缩小空间尺寸, 但保持深度增长的想法。

小卷积核的优势

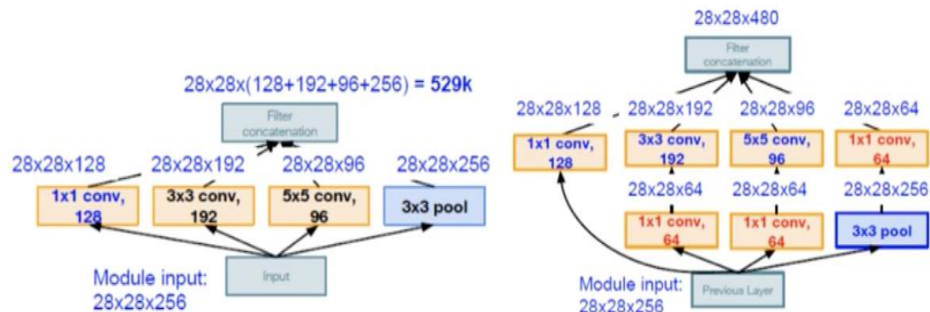
- 更多的激活函数、更丰富的特征, 更强的辨别能力。
- 卷积层的参数减少
- 小卷积核代替大卷积核的正则作用带来性能提升
- 分类精度, 多个小卷积堆叠比单个大卷积好

再次强调CNN必须够深, 视觉数据的层次化表示才有用, 深的同时结构简单!

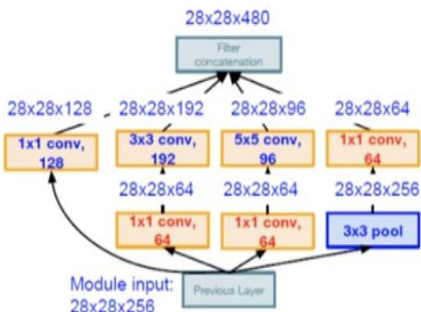


- GoogLeNet: ILSVRC 2014比赛使用此名字, 获得冠军 (6.7%Top 5)
- 增加深度: 22层网络, 在不同深度处增加了两个loss来保证无梯度回传消失现象
- Inception结构: 增加宽度, 用一些1*1和5*5的小卷积核以固定方式组合, 代替大卷积核, 可增加感受野且减少参数。
- 核心思想: 通过多个卷积核提取图像不同尺度的信息, 最后进行融合, 可以得到图像更好的表征。

500万参数, 比AlexNet小12倍



最初的Inception结构



改进的Inception结构

梯度是如何“回传”的?

总损失函数 (Total Loss) 被定义为:

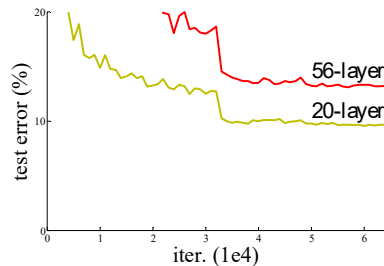
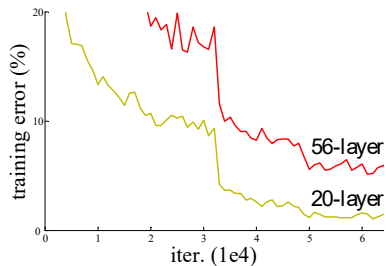
$$L_{total} = L_{final} + 0.3 \times L_{aux1} + 0.3 \times L_{aux2}$$

- 注入梯度: 由于辅助分类器离输入层更近, 它们的梯度可以直接传回较低的层, 而不需要经过漫长的深层路径。
- 权重系数: 为了不干扰主分类器的判断, 辅助分类器的损失权重被设为 0.3。
- 防止过拟合: 除了缓解梯度消失, 这两个分支还起到了类似正则化的作用, 增强了网络的判别能力。



- 2015年提出，在ImageNet比赛classification任务上获得第一名
- “简单与实用”并存，之后很多方法都建立在ResNet50或者ResNet101的基础上完成的，检测、分割、识别等领域都纷纷使用ResNet，Alpha zero也使用了ResNet，至今应用广泛。

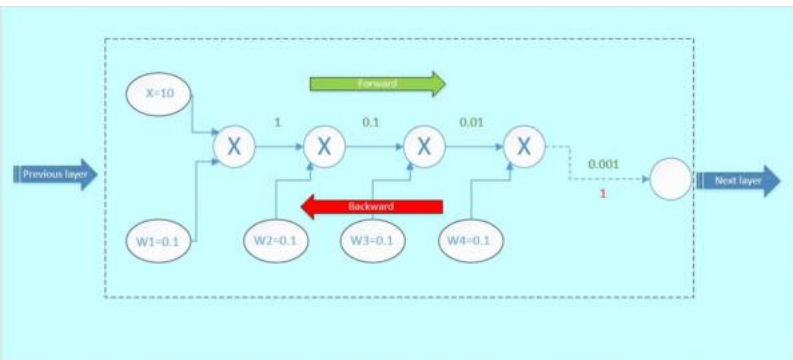
随着网络层级的不断增加，模型精度不断得到提升，而当**网络层级增加到一定的数目以后，训练精度和测试精度迅速下降**，说明当网络变得很深以后，深度网络就变得更加难以训练了。**训练神经网络的反向传播，导致很容易出现梯度消失。**



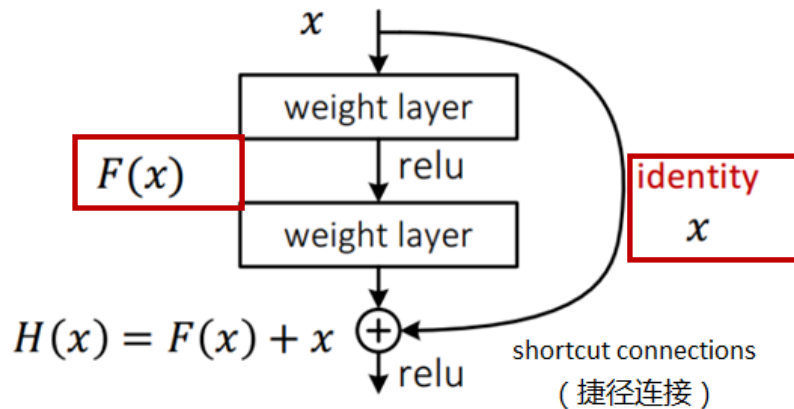
56层比
20层效
果差！

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com



- 假设现有一个比较浅的网络已达到了饱和的准确率，这时在它后面再加上几个恒等映射层（Identity mapping，也即 $y=x$ ，输出等于输入），这样增加了网络的深度，并且误差不会增加，也即更深的网络不应该带来训练集上误差的上升。



直接向下传递的网络：从 x 中直接学习残差 $F(x)$

捷径网络：输入 x

整合：将残差和 x 相加，即 $H(x)=F(x)+x$ ，也就是所要求的映射 $H(x)$

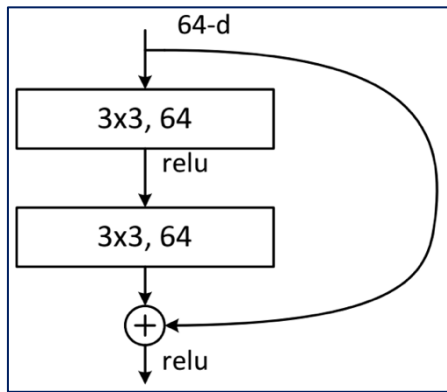
只有一条通路反向传播，会做连乘导致梯度消失，但现在有两条路，会变成求和的形式，避免梯度消失。前向推导时，后面的层可以看见输入，不至于因为信息损失而失去学习能力。

学习的目标：**原始输入与目标输出之间的差值**（目标值 $H(x)$ 和输入 x 的差值，即 $F(x):=H(x)-x$ ），将残差逼近于0，使得随着网络加深，准确率不下降。



两种残差结构

ResNet34

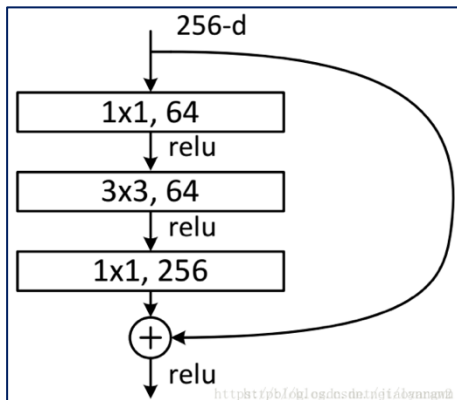


参数个数

左图 $3 \times 3 \times 256 \times 256 + 3 \times 3 \times 256 \times 256 = 1,179,648$

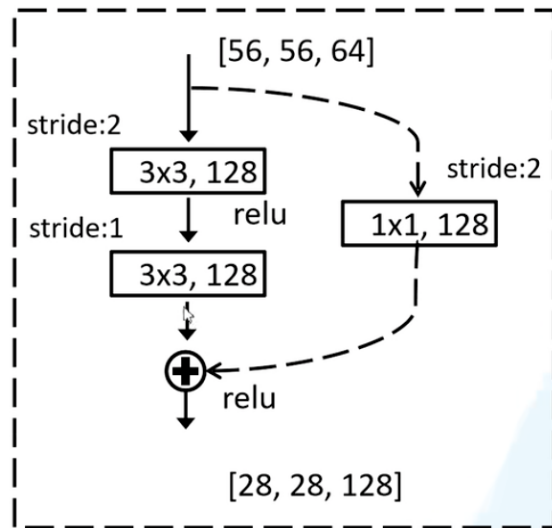
右图 $1 \times 1 \times 256 \times 64 + 3 \times 3 \times 64 \times 64 + 1 \times 1 \times 64 \times 256 = 69,632$

ResNet50/101/152



Bottleneck设计

Option B



主分支与shortcut的输出
特征图大小必须相同



Batch Normalization

Batch Normalization的目的是使我们的一批（Batch）
feature map满足均值为0，方差为1的分布规律。



μ σ^2 在正向传播过程中统计得到

γ β 在反向传播过程中训练得到

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

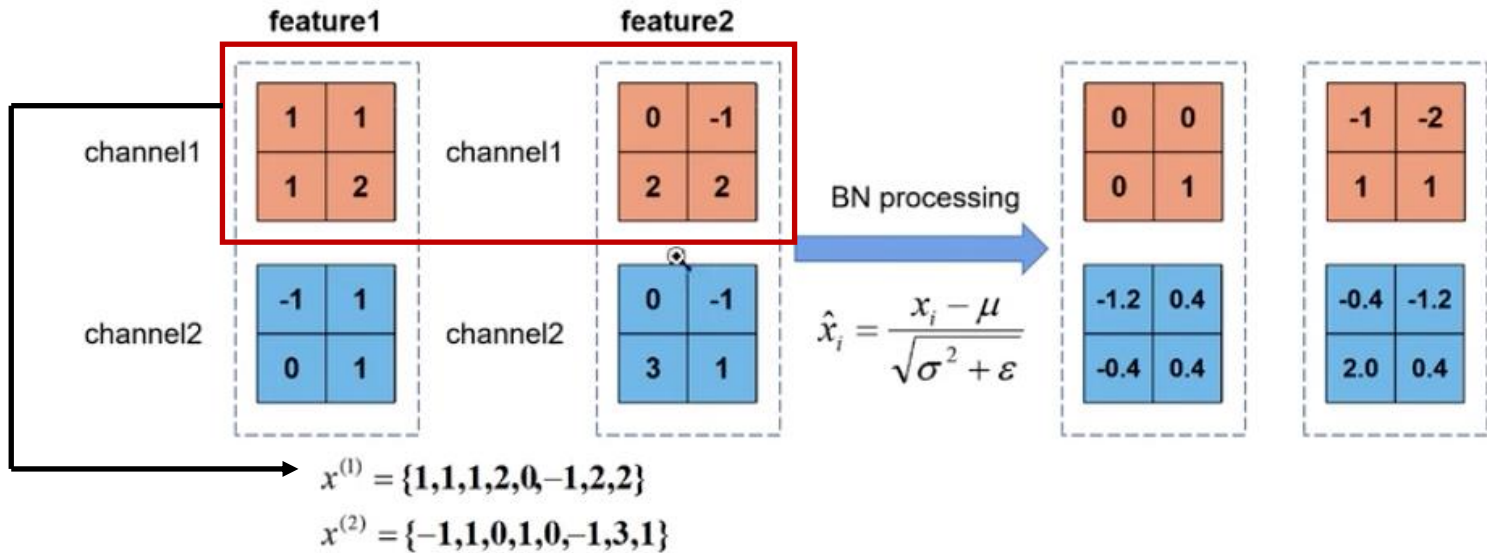
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to
activation x over a mini-batch.



Batch Normalization 例子



$$\mu_1 = \frac{1}{m} \sum_{i=1}^m x_i^{(1)} = 1$$

$$\mu_2 = \frac{1}{m} \sum_{i=1}^m x_i^{(2)} = 0.5$$

$$\sigma_1^2 = \frac{1}{m} \sum_{i=1}^m (x_i^{(1)} - \mu_1)^2 = 1$$

$$\sigma_2^2 = \frac{1}{m} \sum_{i=1}^m (x_i^{(2)} - \mu_2)^2 = 1.5$$



$$\mu = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} \quad \sigma^2 = \begin{bmatrix} 1 \\ 1.5 \end{bmatrix}$$



什么是梯度消失 (Vanishing Gradient Problem)?

简单来说，**梯度消失**是指在深度神经网络的训练过程中，靠近输入层的参数权重更新极其缓慢，甚至完全停滞。

在反向传播 (Backpropagation) 中，误差会从输出层一层层向输入层传递。梯度就像是“指挥棒”，告诉网络参数该往哪个方向调。如果这根指挥棒在传递过程中不断“缩水”，传到最前面时已经近乎为零，那么前层的神经元就学不到任何东西，网络也就无法收敛。



1. 核心诱因：链式法则的连乘效应

神经网络的参数更新依赖于链式法则。假设我们有一个简单的 n 层网络，每一层的激活函数为 f ，权重为 w 。在计算第一层参数的梯度时，我们需要连乘后面每一层的导数。

如果每一层的导数（激活函数的导数与权重的乘积）都小于 1，那么经过 n 层连乘后，结果会以指数级速度衰减。

$$\frac{\partial Loss}{\partial w_1} = \frac{\partial Loss}{\partial out} \times \dots \times \frac{\partial out_2}{\partial out_1} \times \frac{\partial out_1}{\partial w_1}$$

如果每一项都是 0.1，那么 10 层之后，梯度就会变成 $0.1^{10} = 0.0000000001$ 。

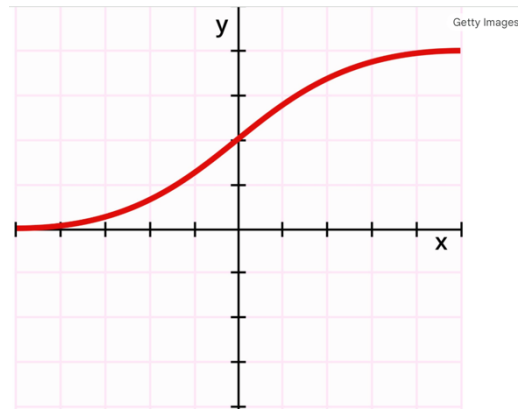




2. 举例说明：Sigmoid 激活函数的“锅”

在早期的深度学习中，**Sigmoid** 是最常用的激活函数。它是梯度消失现象的“典型代表”。

- **Sigmoid 函数公式：** $\sigma(x) = \frac{1}{1 + e^{-x}}$
- **它的导数：** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$



为什么它会导致梯度消失？

观察 Sigmoid 导数的图像你会发现，它的**最大值仅为 0.25**（当 $x = 0$ 时）

1. 如果你的输入 x 很大或很小（进入饱和区），导数会趋近于 **0**。
2. 即使输入都在理想范围内，每经过一层 Sigmoid 网络，梯度至少要**萎缩 75%**（乘以 ≤ 0.25 的数）。





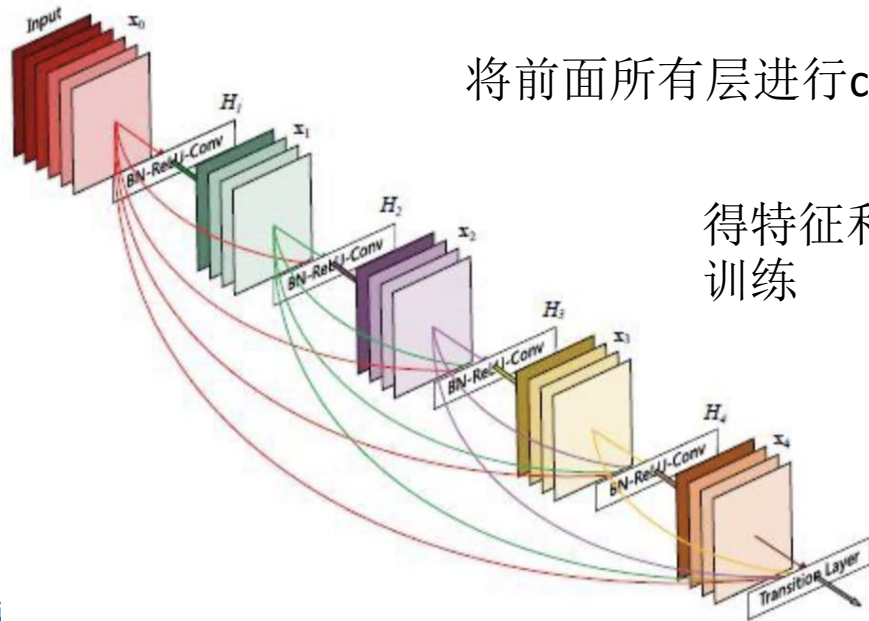
3. 如何解决梯度消失？

为了应对这个问题，深度学习领域演化出了多种技术：

- **更换激活函数**：使用 **ReLU** ($f(x) = \max(0, x)$)。ReLU 在 $x > 0$ 时导数始终为 **1**，完美解决了连乘衰减问题。
- **归一化层 (Batch Normalization)**：强行将每一层的输入拉回到导数较大的区域，避免进入激活函数的饱和区。
- **残差结构 (ResNet)**：通过“捷径” (Shortcut) 让梯度可以跨层直接流动。
- **辅助损失函数 (Auxiliary Loss)**：正如你在 GoogLeNet 中看到的，在中间层直接接入 Loss，给前层网络提供“短路”式的梯度信号。



核心思想：保证网络中层与层之间最大程度的信息传输的前提下，直接将所有层连接起来！



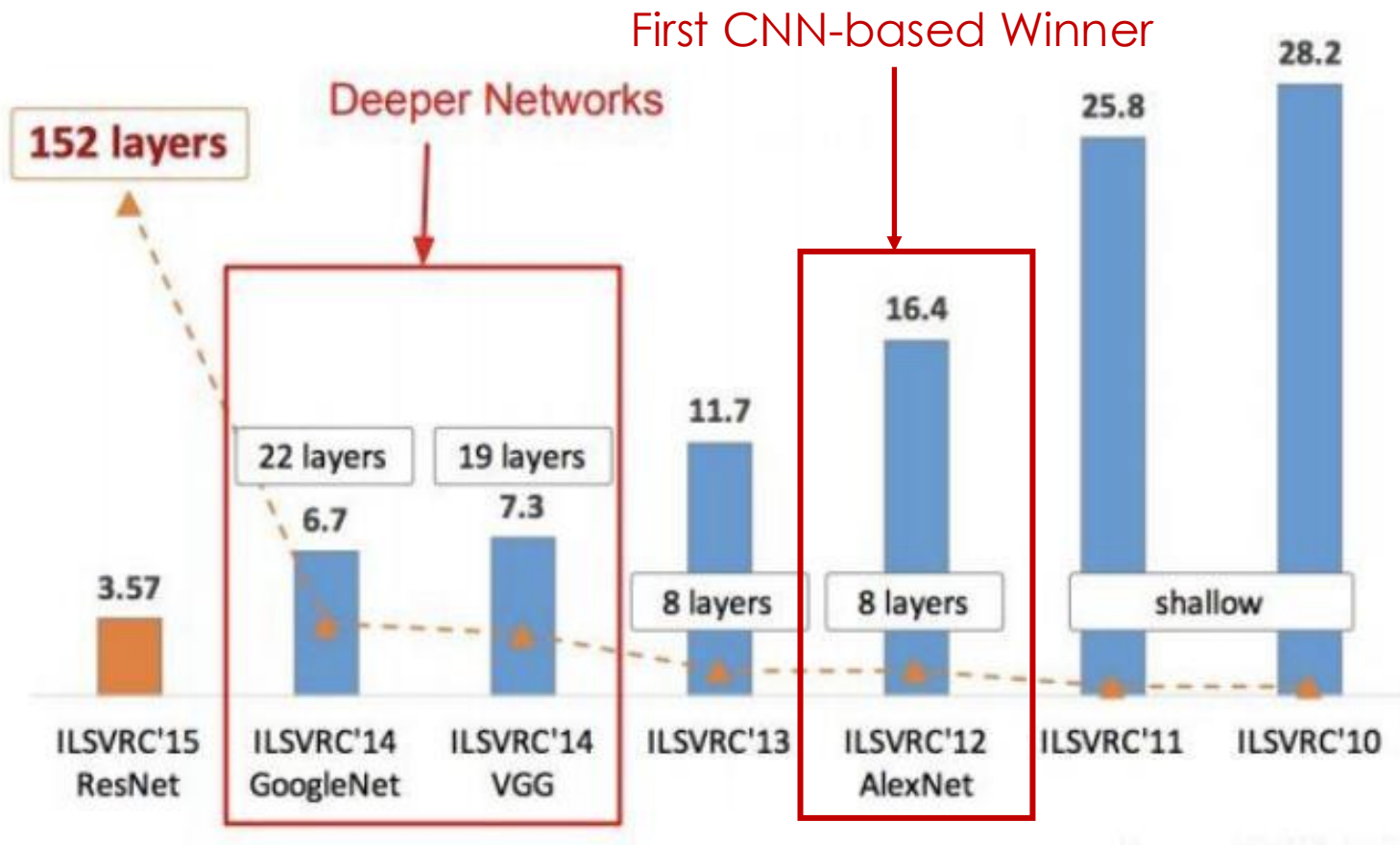
将前面所有层进行concatenation操作，即做通道的合并

得特征和梯度的传递更加有效，网络也就更加容易训练

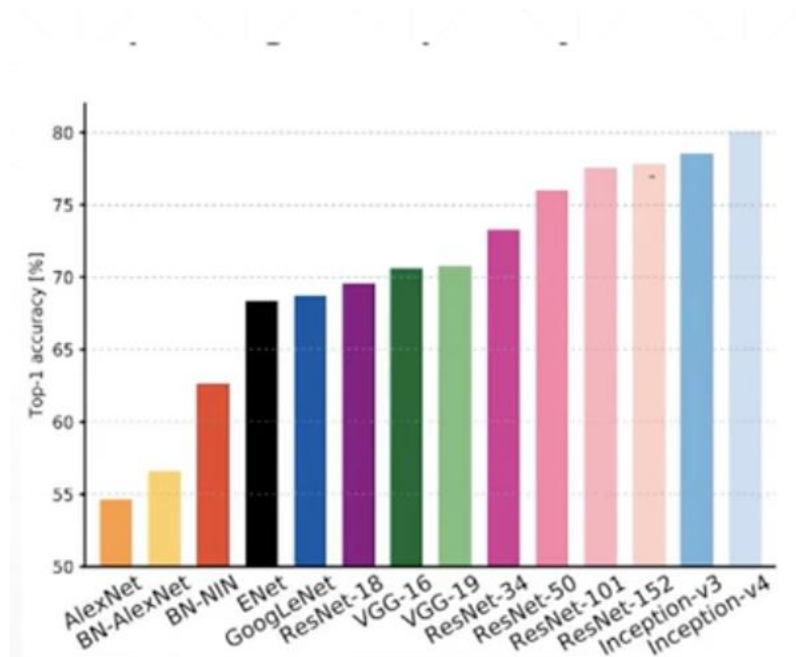
网络变窄，参数减少，有效抑制了过拟合，同时减少计算量



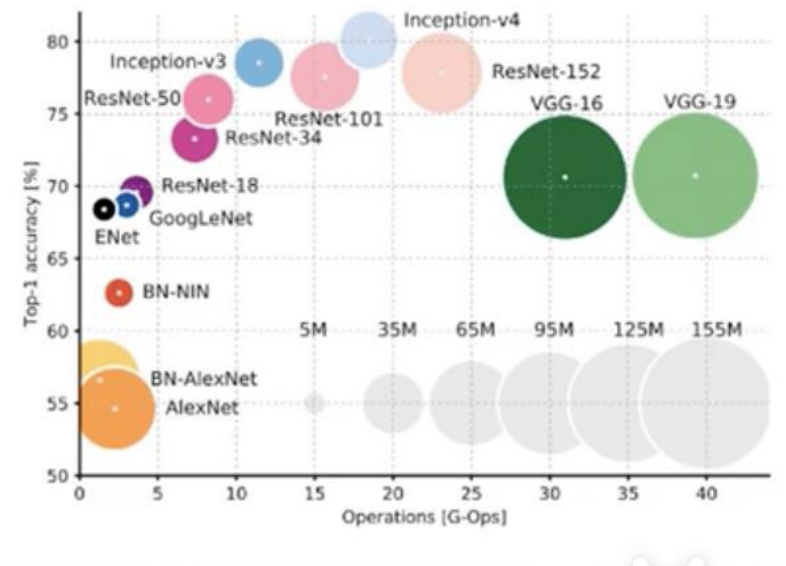
比较不同网络



比较不同网络



不同网络准确率比较



不同网络计算量与准确率

- 深度学习基本概述
- 卷积神经网络原理
- 常见的卷积神经网络结构
- 目标检测与分割



计算机视觉领域的任务

- 图像分类
- 目标检测
- 图像分割
- 目标跟踪
- 图像滤波与降噪
- 图像增强
- 风格迁移
- 三维重建
- 图像检索
- 图像生成



图像获取

提取二维图像、三维图组、图像序列或相关的物理数据，如声波、电磁波或核磁共振的深度、吸收度或反射度



预处理

对图像做一种或一些预处理，使图像满足后继处理的要求，如：二次取样保证图像坐标的正确，平滑、去噪等



特征提取

从图像中提取各种复杂度的特征，如：线，边缘提取和脊检测，边角检测、斑点检测等局部化的特征点检测



检测/分割

对图像进行分割，提取有价值的内容，用于后继处理，如：筛选特征点，分割含有特定目标的部分



高级处理

验证得到的数据是否匹配前提要求，估测特定系数，对目标进行分类





目标检测与分割

Classification



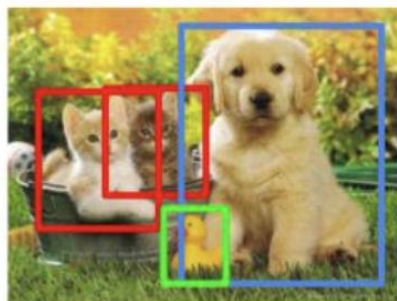
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



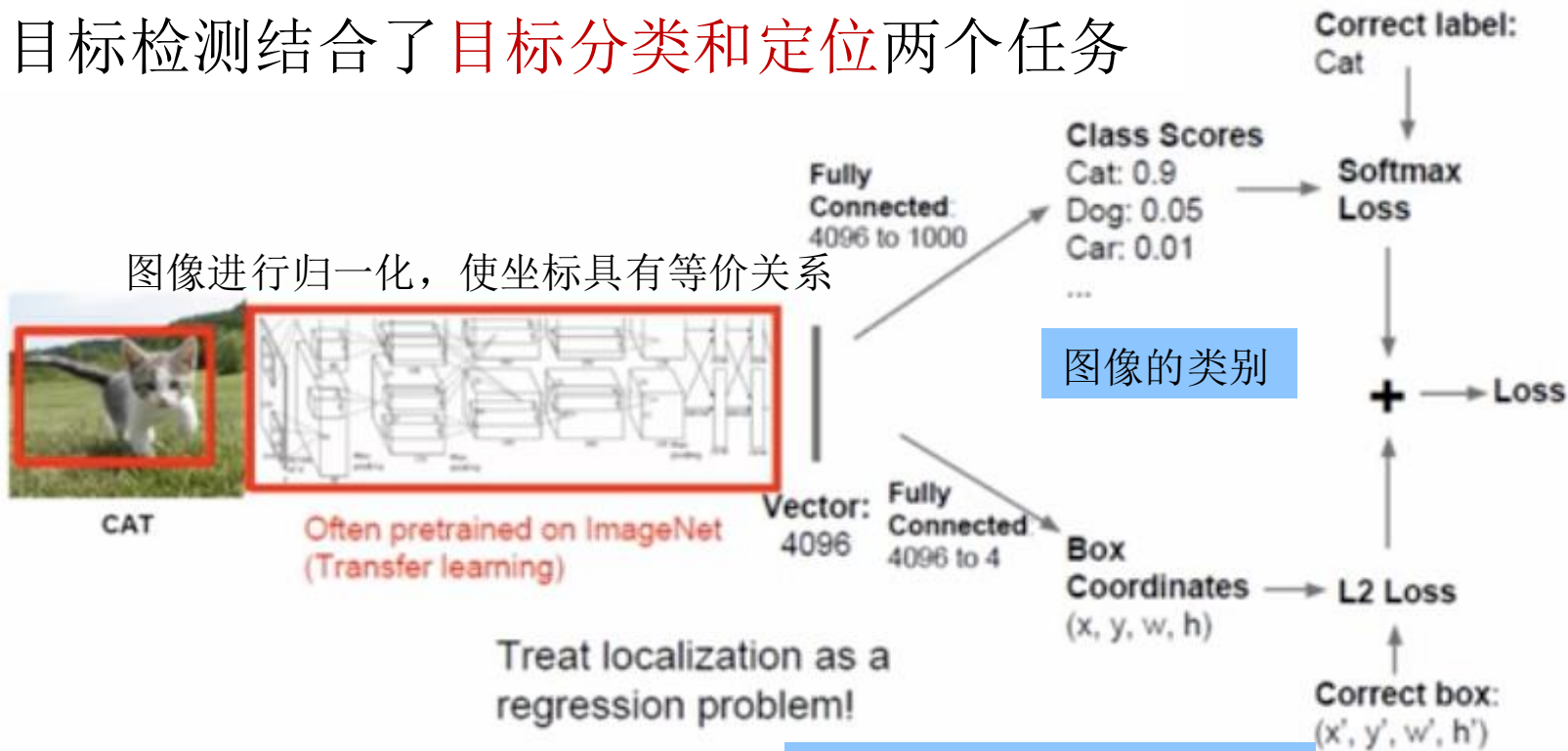
CAT, DOG, DUCK

Single object

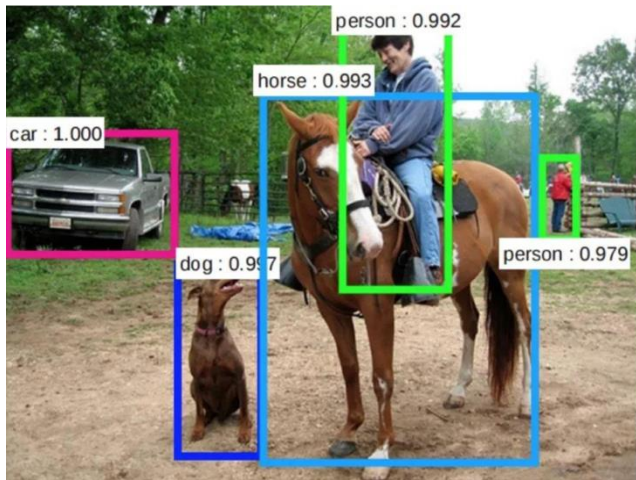
Multiple objects

难度逐渐升级

目标检测结合了目标分类和定位两个任务



最后一层加4个维度的输出，
左上坐标点和方框的长宽



目标检测：图像中有多个目标，检测其位置，并对每个目标要进行识别

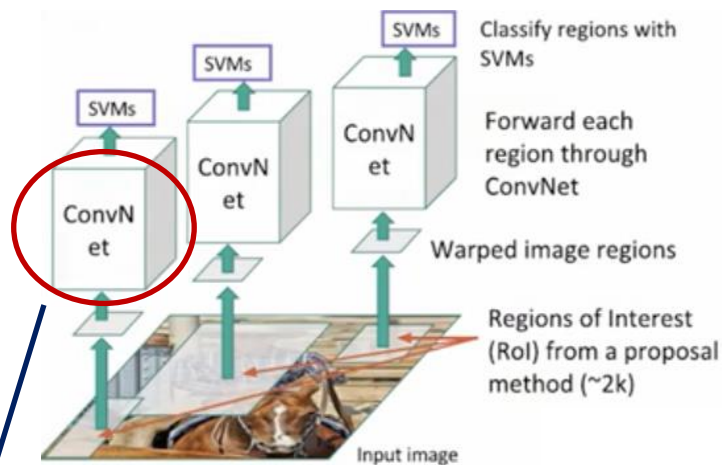
关键问题：不知道有多少个目标，以及目标在哪里，用大大小小的方框遍历所有图像不现实。

Two-stage算法：基于Region Proposal的R-CNN系算法，提出候选框，即目标位置，再对候选框做分类与回归

One-stage算法：使用一个卷积神经网络CNN直接预测不同目标的类别与位置。



R-CNN 采用的是 Selective Search 算法，通过一些传统图像处理方法将图像分成很多小尺寸区域，然后根据小尺寸区域的特征（相似度判断）合并小尺寸得到大尺寸区域，以实现候选区域的选取。



- (1) 采用selective search提取个候选框RoI
- (2) 对候选框进行长宽归一化，输入CNN网络，获得目标识别结果（CNN用于特征提取）
- (3) 使用SVM判决候选区域是否存在目标

R-CNN缺点：计算量非常大，每个图像selective search 耗时5s，将产生2000个左右候选区域，每个候选区域输入CNN，每个候选区都输入SVM，处理一个图片要1分钟。

AlexNet

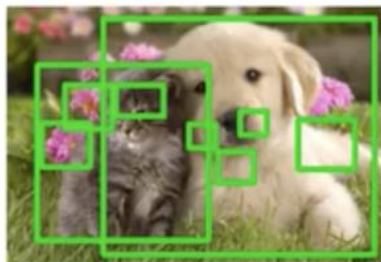
Ross B. Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik:

Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. CVPR 2014: 580-587



Fast RCNN (1)

- 核心思想：ROI-Pooling，解决了重复卷积操作问题，加速CNN特征提取过程



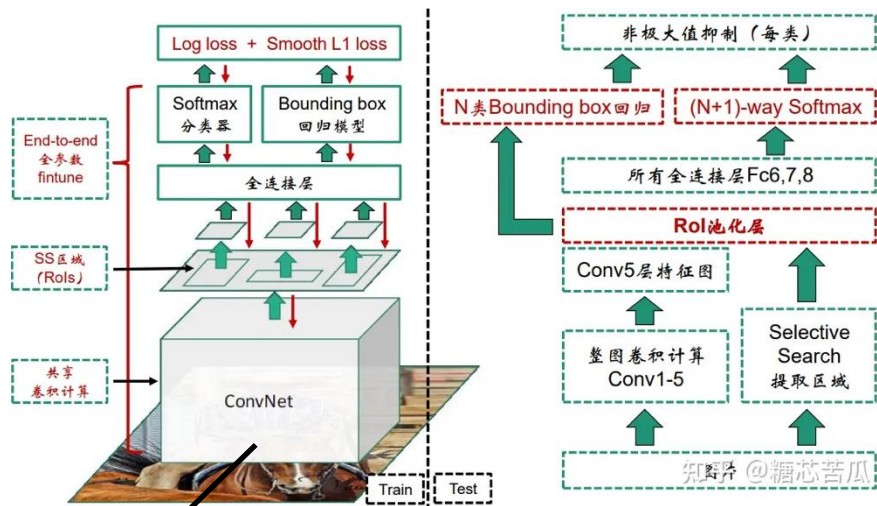
重叠区域输入CNN中，导致很多重复的卷积操作

- (1) Fast R-CNN通过CNN直接获取整张图像的特征图
- (2) 在中间某一层特征图上，使用**RoI Pooling**，获取对应每个候选框的特征，避免了R-CNN中的对每个候选框串行进行卷积
- (3) 分别经过为21和84维的全连接层（并列，用于分类和回归的输出）

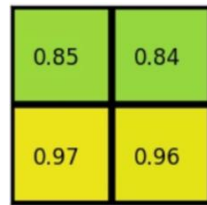
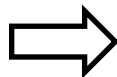
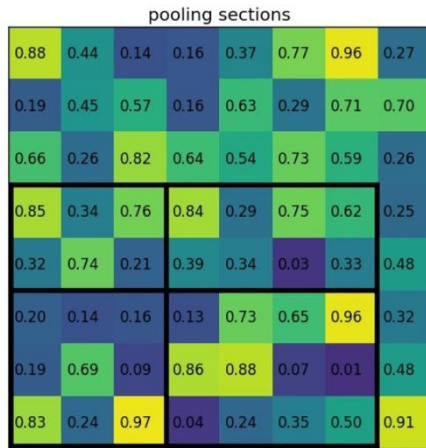


Fast RCNN (2)

ROI Pooling: 为每个RoI选取对应的特征; 为了满足全连接层的输入需求, 将每个RoI对应的特征的维度转化成某个定值



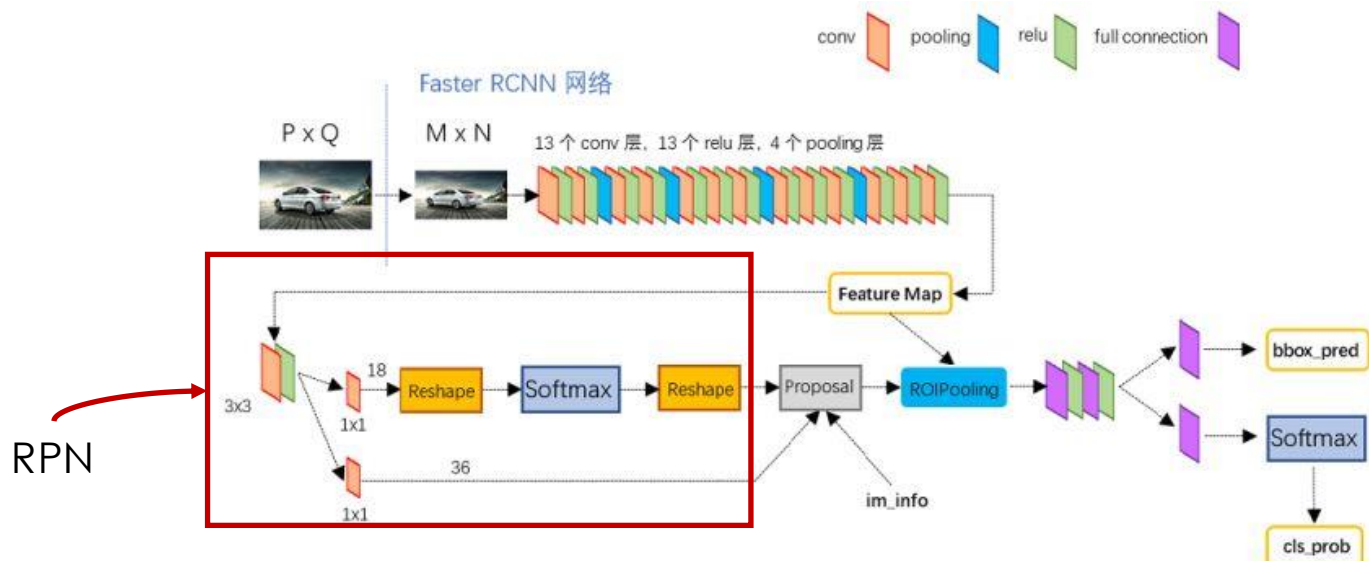
先用CNN特征提取



- 将region proposal划分为目标 $H \times W$ 大小的分块
- 对每一个分块中做MaxPooling (每个分块中含有多个网格, 每个分块获取一个特征值)
- 将所有输出值组合起来便形成固定大小为 $H \times W$ 的feature map

耗时的selective search依旧存在!

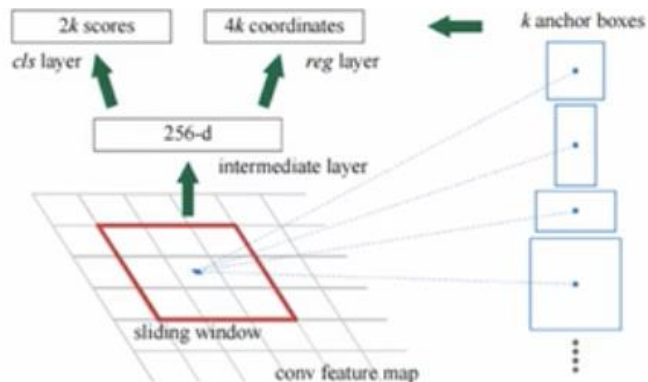
Faster R-CNN由共享卷积层、RPN、RoI pooling以及分类和回归四部分组成



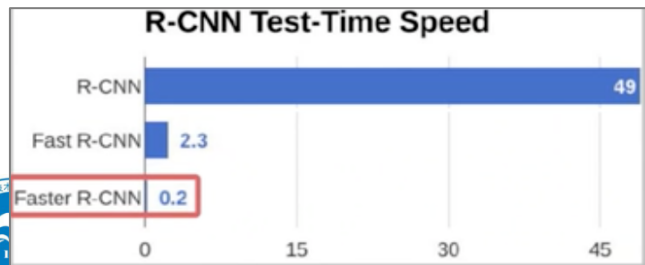
- (1) 使用共享卷积层为全图提取特征图
- (2) 在特征图上滑动窗口，用不同长宽比的矩形作为候选区域
- (3) 用一个小网络判断这些候选框是否存在目标
- (4) 运用ROI Pooling归一化

Faster RCNN (2)

核心思想：取代selective search，直接通过一个**Region Proposal Network (RPN)**生成待检测区域，将生成RoI区域的时间也就从2s缩减到了10ms。

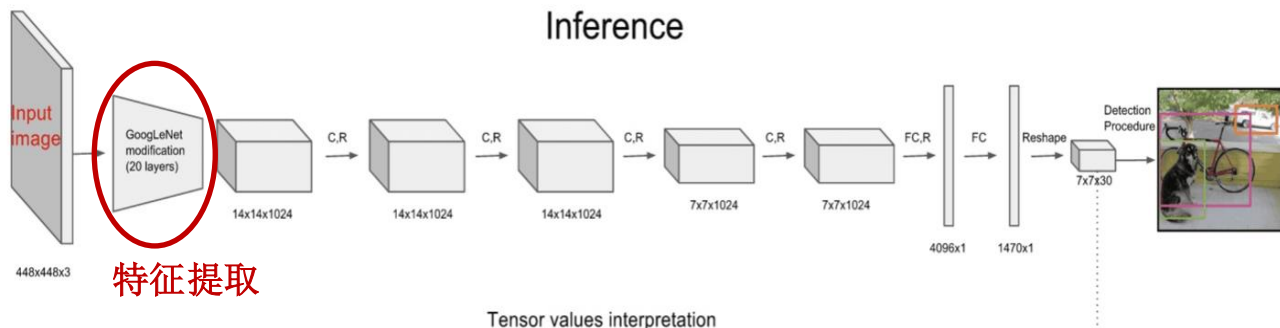


Anchor: RPN依靠一个在共享特征图上滑动的窗口，为每个位置生成**9种预先设置好长宽比与面积的目标框**(即anchor)。这9种初始anchor包含三种面积(128×128 , 256×256 , 512×512), 每种面积又包含三种长宽比(1:1, 1:2, 2:1)。通过anchor引入了检测中常用的多尺度方法。



Faster R-CNN具备了实时处理图像的能力

- Yolo创造性的提出了 **one-stage**，将物体分类和物体定位在一个步骤中完成。
- 将图像分为 $s*s$ 个格子，某个物体的ground truth坐标落入某个格子中。
- **Yolo直接在输出层回归bounding box的位置和bounding box所属类别**，从而实现one-stage。通过这种方式，Yolo可实现45帧每秒的运算速度，完全能满足实时性要求（达到24帧每秒，人眼就认为是连续的）



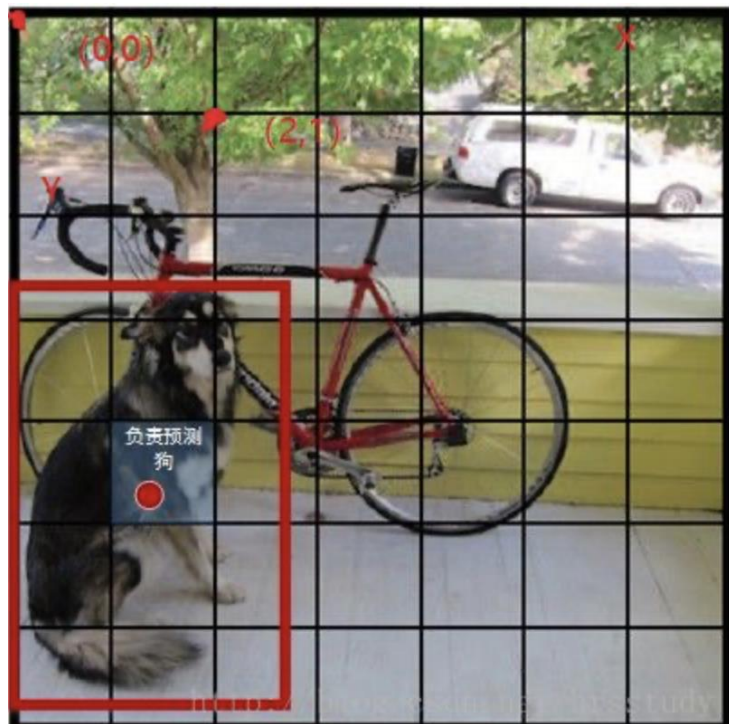
特征提取

卷积层、目标检测层、NMS筛选层

• Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, Ali Farhadi:
You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016: 779-788



YOLO (2)



Yolo将一副448x448的原图分割成了7x7个网格，然后每个单元格负责去检测那些中心点落在该格子内的目标

每个单元格会预测B个bounding box以及边界框的置信度 (confidence score)

置信度包含两个方面：这个边界框含有目标的可能性大小；这个边界框的准确度

YOLOv11: AN OVERVIEW OF THE KEY ARCHITECTURAL ENHANCEMENTS

Rahima Khanam* and Muhammad Hussain

Department of Computer Science, Huddersfield University, Queensgate, Huddersfield HD1 3DH, UK;
*Correspondence: rahima.khanam@hud.ac.uk;

October 24, 2024

ABSTRACT

This study presents an architectural analysis of YOLOv11, the latest iteration in the YOLO (You Only Look Once) series of object detection models. We examine the models architectural innovations, including the introduction of the C3k2 (Cross Stage Partial with kernel size 2) block, SPPF (Spatial Pyramid Pooling - Fast), and C2PSA (Convolutional block with Parallel Spatial Attention) components, which contribute in improving the models performance in several ways such as enhanced feature extraction. The paper explores YOLOv11's expanded capabilities across various computer

目前最新
YOVO版
本V11，
2024



YOLO: 简史

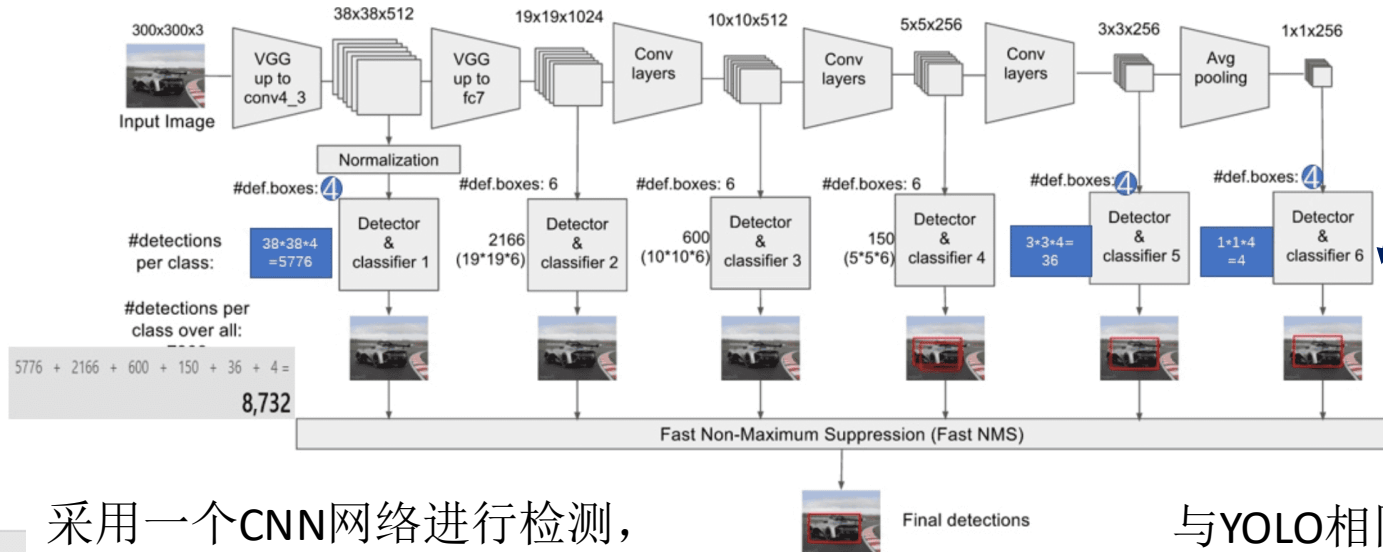
YOLO (You Only Look Once), 由华盛顿大学的Joseph Redmon和Ali Farhadi开发的流行目标检测和图像分割模型, 于2015年推出, 由于其高速和准确性而迅速流行。

- **YOLOv2** 在2016年发布, 通过引入批量归一化、锚框和维度聚类来改进了原始模型。
- **YOLOv3** 在2018年推出, 进一步增强了模型的性能, 使用了更高效的主干网络、多个锚点和空间金字塔池化。
- **YOLOv4** 在2020年发布, 引入了Mosaic数据增强、新的无锚检测头和新的损失函数等创新功能。
- **YOLOv5** 进一步改进了模型的性能, 并增加了新功能, 如超参数优化、集成实验跟踪和自动导出到常用的导出格式。
- **YOLOv6** 在2022年由**美团**开源, 现在正在该公司的许多自动送货机器人中使用。
- **YOLOv7** 在COCO关键点数据集上添加了额外的任务, 如姿态估计。
- **YOLOv8** 是Ultralytics的YOLO的最新版本。作为一种前沿、最先进(SOTA)的模型, YOLOv8在之前版本的成功基础上引入了新功能和改进, 以提高性能、灵活性和效率。YOLOv8支持全范围的视觉AI任务, 包括**检测**, **分割**, **姿态估计**, **跟踪**, 和**分类**。这种多功能性使用户能够利用YOLOv8的功能应对多种应用和领域的需求。

<https://blog.csdn.net/java1314777/article/details/134097996>



- Faster R-CNN准确率mAP较高，漏检率recall较低，但速度较慢。而Yolo则相反，速度快，但准确率和漏检率不尽人意
- SSD综合了他们的优缺点，对输入300x300的图像，在voc2007数据集上达到58 帧每秒(Titan X 的 GPU)且72.1%的mAP。



每一个卷积层，
都输出不同大小
感受野的特征图



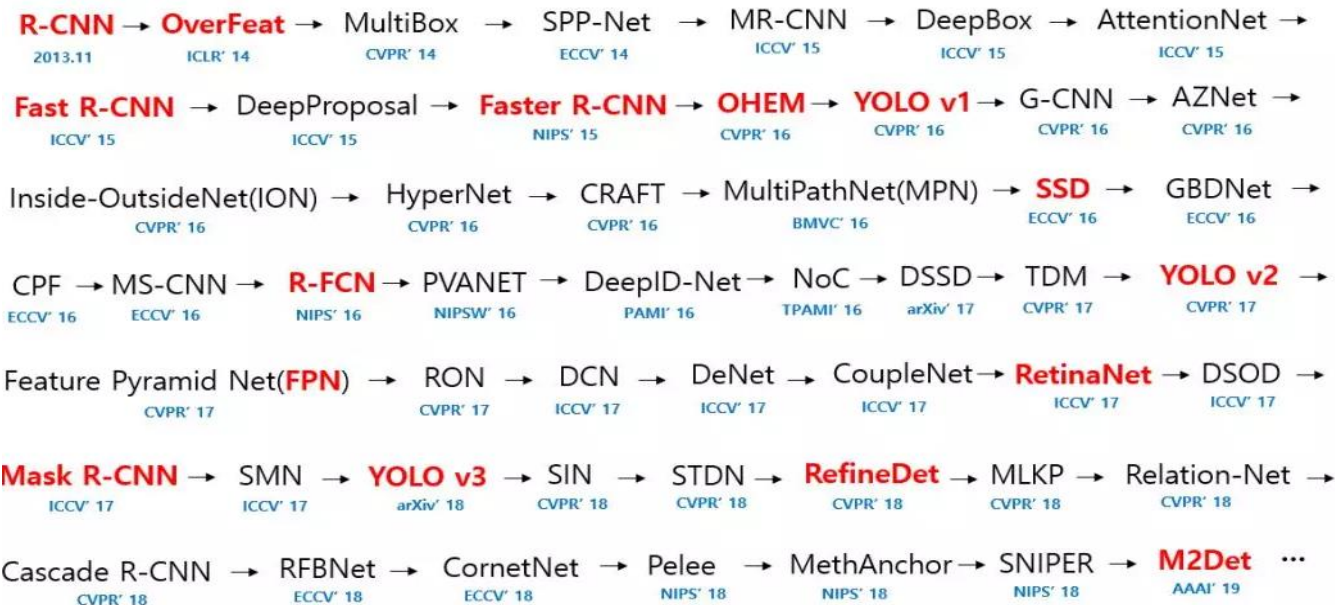
采用一个CNN网络进行检测，
采用了多尺度的特征图

与YOLO相同，包含卷积层、
目标检测层和NMS筛选层



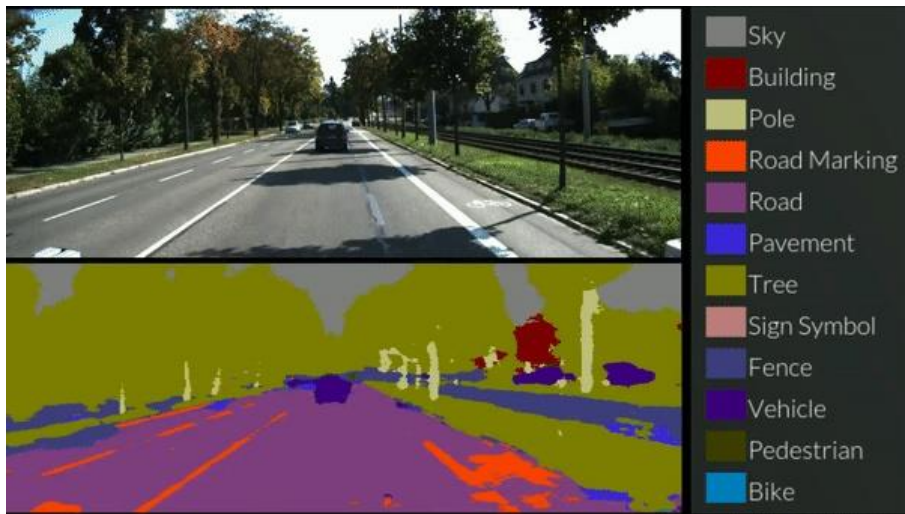
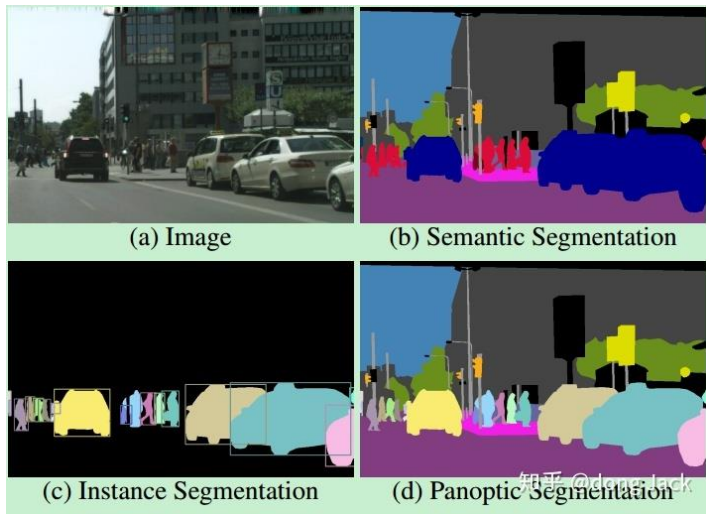
one-stage(YOLO,YOLO9000,YOLOV3,..., YOLOV7、 SSD等)

two-stage(OverFeat, R-CNN, Fast R-CNN, Faster R-CNN 等)





图像分割



- ✓ **普通分割**：将不同分属不同物体的像素区域分开。如前景与背景分割开，狗的区域与猫的区域与背景分割开。
- ✓ **语义分割**：在普通分割的基础上，分类出每一块区域的语义（即这块区域是什么物体），如把画面中的所有物体都指出它们各自的类别。
- ✓ **实例分割**：在语义分割的基础上，给每个物体编号，如这个是该画面中的狗A，那个是画面中的狗B。

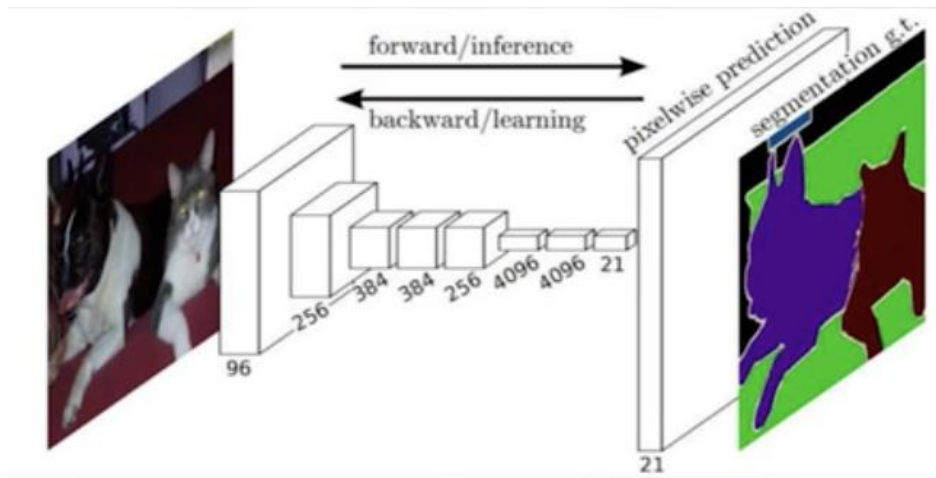
基本思路：

- 下采样+上采样：Convolution + Deconvolution / Resize
- 多尺度特征融合：特征逐点相加 / 特征channel维度拼接
- 获得像素级别的segment map：对每一个像素点进行判断类别



全卷积网络FCN

FCN: 全卷积网络, 用于语义分割



输出是与输入一样长宽的图像

特征图先缩小再放大, 中心对称结构

关键点: 特征图的放大, 采用**上采样方法**



Pooling层的上采样 (Upsampling)

(a) Average pooling

Nearest Neighbor

1	2
3	4

Input: 2 x 2

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

"Bed of Nails"

1	2
3	4

Input: 2 x 2

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

(b) Max pooling

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

Rest of the network

Max Unpooling
Use positions from
pooling layer

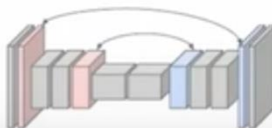
1	2
3	4

Input: 2 x 2

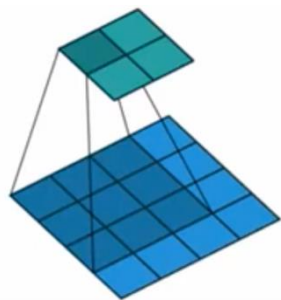
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

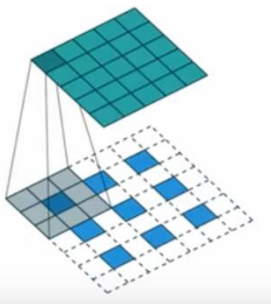
Corresponding pairs of
downsampling and
upsampling layers



卷积层的上采样也称为反卷积（Deconvolution）或转置卷积（Transpose Convolution）

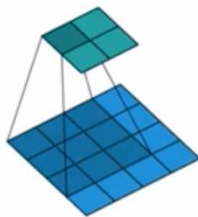


输入特征图4*4，卷积核3*3，步长1，卷积后获得特征图为2*2



输入特征图5*5，卷积核3*3，步长2，补零1，卷积后获得特征图维度为3*3，反卷积示意图

卷积流程

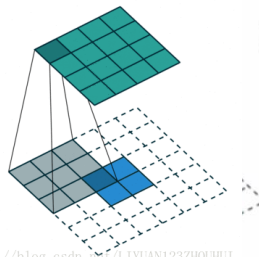


$$C = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 \end{pmatrix}$$

$$X = [x_1, x_2, \dots, x_{16}]^T$$

$$Y = CX$$

反卷积流程



$$C = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 \end{pmatrix}$$

$$X = [x_1, x_2, \dots, x_{16}]^T$$

$$X = C^T Y$$



Unet, 2015年提出, 最初用于医学图像分割, 目前可用在其他分割任务, 效果超过FCN

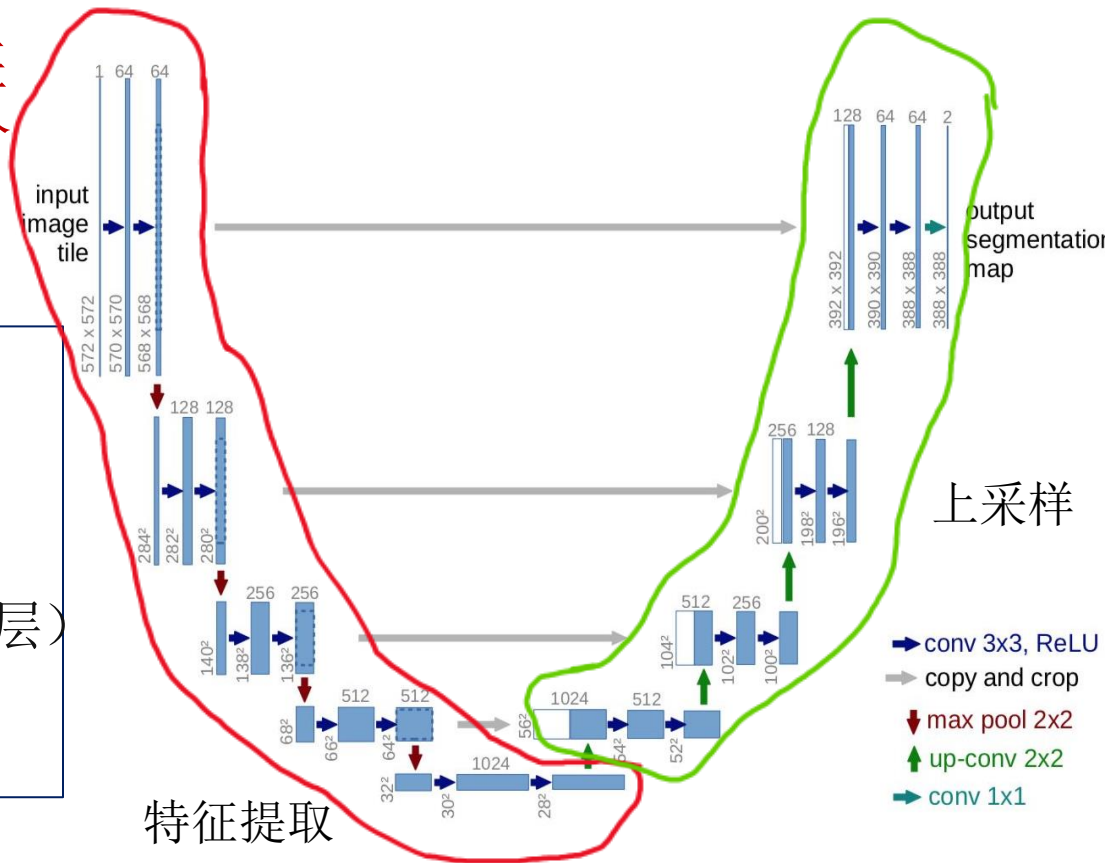
Encoder-Decoder的结构

➤ 前半部分特征提取

- 2个3x3的卷积层 (ReLU)
- 1个2x2的最大池化层

➤ 后半部分上采样

- 上采样的卷积层 (反卷积层)
- 特征拼接concat
- 2个3x3的卷积层 (ReLU)



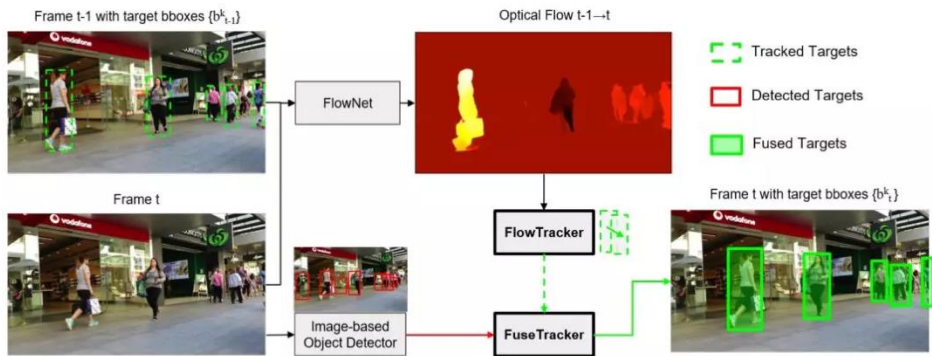
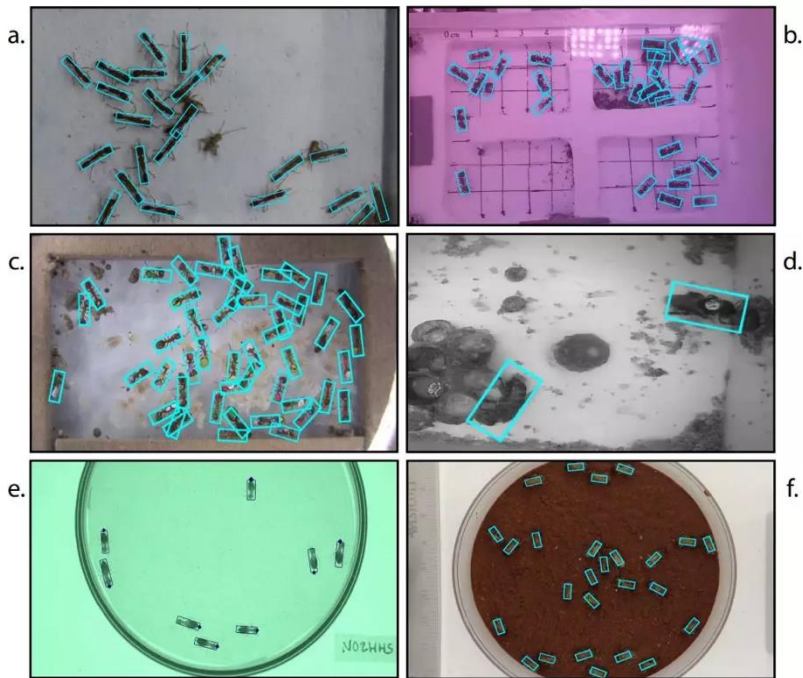


Figure 2. The overall view of Flow-Fuse-Tracker (FFT) for multiple object tracking. FlowTracker and FuseTracker (in bold grey boxes) are the two DNN modules of our FFT network. In the FlowTracker, the optical flow is generated from two sequential frames, and the target bboxes $\{b_{t-1}^k\}$ (green dashed bboxes) at frame $t-1$ are regressed to the bboxes $\{b_t^k\}$ at frame t through the optical flow. In the FuseTracker, the bboxes from both $\{b_t^k\}$ and public detections D_t (red bboxes) at frame t are refined and fused. The FuseTracker outputs the final tracking results (green bboxes with shadow). (Best viewed in color)

Segment Anything Model (SAM), Meta 2023.4

大模型在CV上突破，单一的模型执行交互式分割和自动分割，未来可应用于更多领域（大的含义不只是参数量），比如AR



单块V100上，对于1200x800这种正常分辨率的图，生成所有的mask需要2-3秒，可提供可用的feature extractor

最终的数据集超过 11 亿个分割掩码

<https://segment-anything.com/dataset/index.html>

分割问题大致有两种方法：

交互式分割：允许分割任何类别的对象，但需要一个人通过迭代细化掩码来指导该方法

自动分割：允许分割提前定义的特定对象类别（例如，猫或椅子），但需要大量的手动注释对象来训练（例如，数千甚至数万个分割猫的例子）。

SAM 是一个单一的模型，可以轻松地执行交互式分割和自动分割。该模型的可提示界面允许用户以灵活的方式使用它，只需为模型设计正确的提示（点击、**boxes**、文本等），就可以完成范围广泛的分割任务。



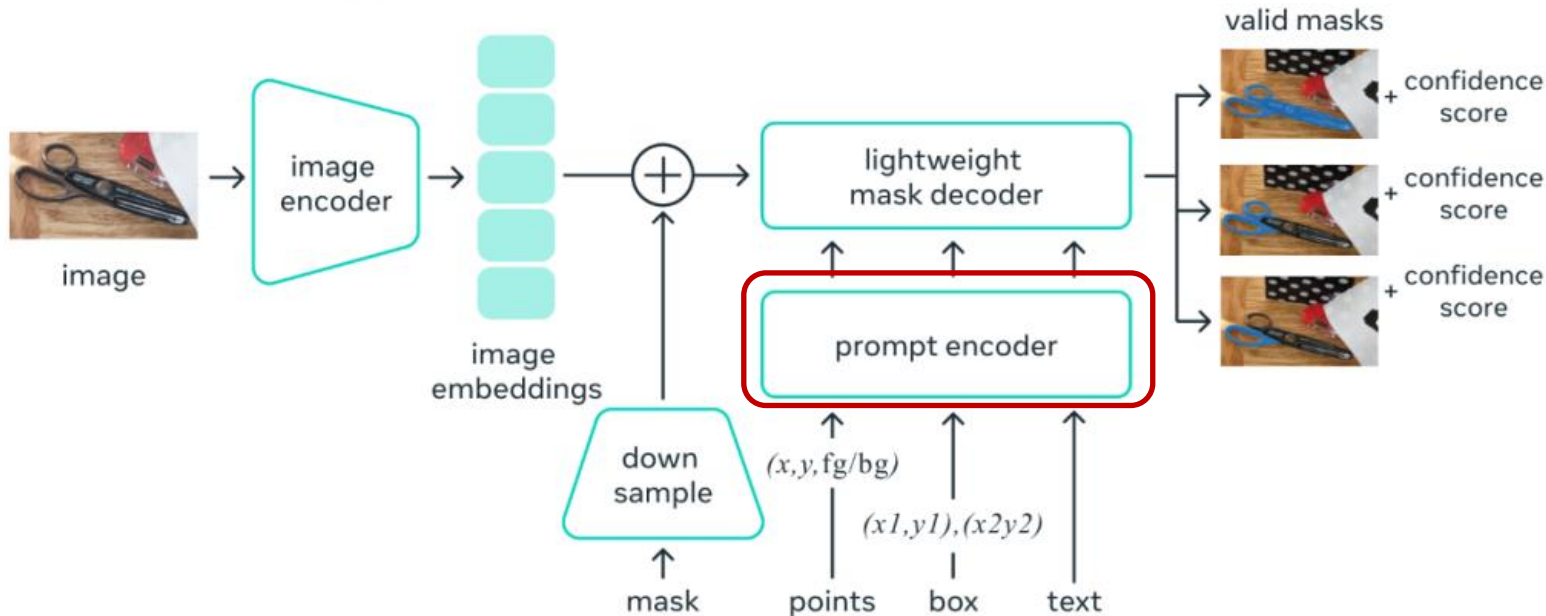
Segment Anything

- 通用涵盖广泛的用例（学会了关于物体的一般概念）。
- 零样本迁移（可以在新的图像领域上即开即用，无需额外的训练）。
- SAM 能很好的自动分割图像中的所有内容。
- SAM 能根据提示词进行图像分割。
- SAM 能用交互式点和框的方式进行提示。
- SAM 能为不明确的提示生成多个有效掩码。可以为任何图像或视频中的任何物体（包括训练中没有的）生成 mask。



文本提示: crossroad
in the city

Universal segmentation model

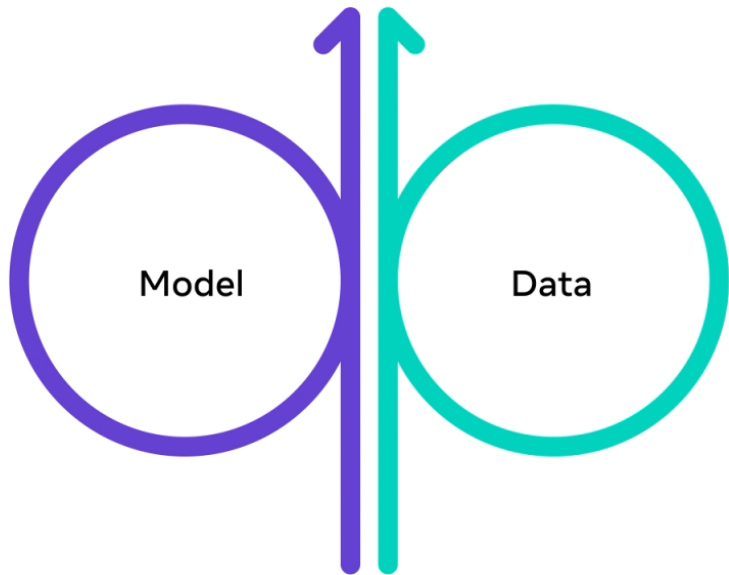


Prompt工程：对新数据集和任务执行零样本学习和少样本学习即可使其能够基于任何提示返回有效的分割掩模。其中，提示可以是前景/背景点、粗略框或掩模、自由文本或者一般情况下指示图像中需要进行分割的任何信息

model-in-the-loop "data engine."

模型与数据一起迭代更新

11M images, 1B+ masks



机器视觉领域

物体检测&场景识别

- 常用物体的目标检测、30000类物体识别。
- 在不同场景投放相关产品的广告
- 在用户心中建立场景与品牌之间的关联与认知。比如沙滩 - 防晒霜，加班 - 功能饮料，餐厅 - 酒水等
- 建立场景 - 明星或场景 - 商品等多维度品牌营销方式



识别物体: 紫叶李~background | 桃金娘科~background | 树
识别场景: 日式花园 | 规则式庭园 | 院子 |



识别物体: 山~background | 山丘~background | 山脉
识别场景: 苔原 | 冰川 | 山 |



识别物体: 卷帘~background | 起居室~background | 床
识别场景: 卧室 | 酒店客房 | 宿舍 |



识别物体: 路标~background | 招牌~background |
识别场景: 建筑外景 | 咖啡店 | 杂货店/室外 |



识别物体: 女旗袍~overcoat | 女体闲套装~upper_cloth | 女礼服
识别场景: 宿舍 | 卧室 | 艺术工作室 |

CVPR iMaterialist 细粒度识别 挑战赛

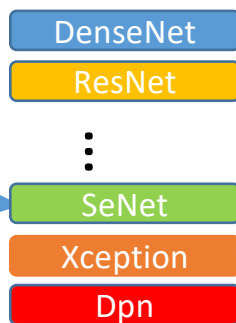
- 是 CVPR 迄今数据规模最大、种类最多的商品识别竞赛，可直接转化为工业界的应用，
- 数据集超过一百万张图像，共涵盖了2019类商品，数据集本身不够准确，韭菜的类别下各种其他蔬菜的错误分类很多



根据评估
结果清洗
数据集



多尺度训练



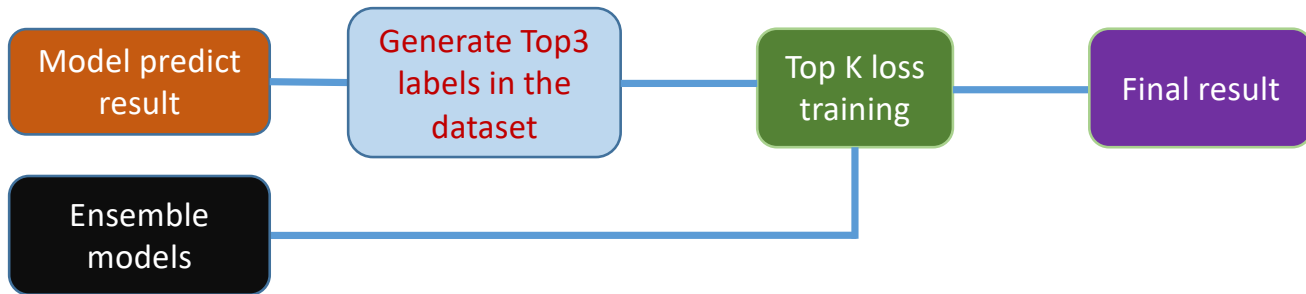
Ensemble
models

Model predict
result

根据网络得到的Score以及投票数量做结果融合

在全球202支参赛队伍中脱颖而出, 勇夺全球第三名!

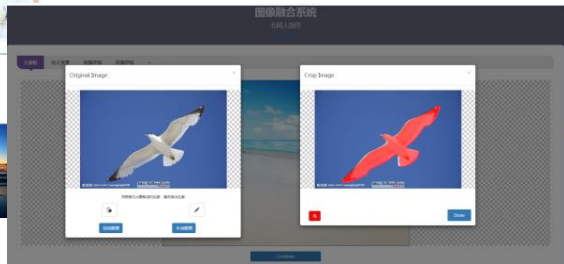
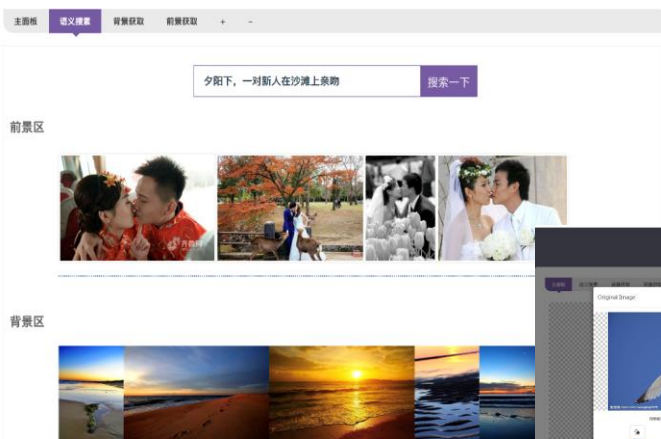
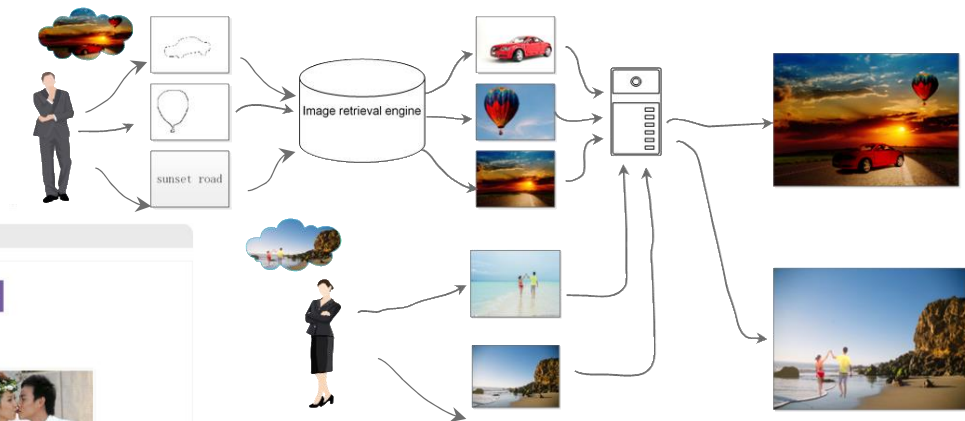
- 细粒度分类很多物体非常相似, 让网络学习到**更多的Top3**标签信息, 用融合后的网络模型**Top3**预测结果, 重新微调每一个单网络结构。重新用较小学习率训练模型, 让模型学到更丰富的**top3**信息, 提升分类准确率。



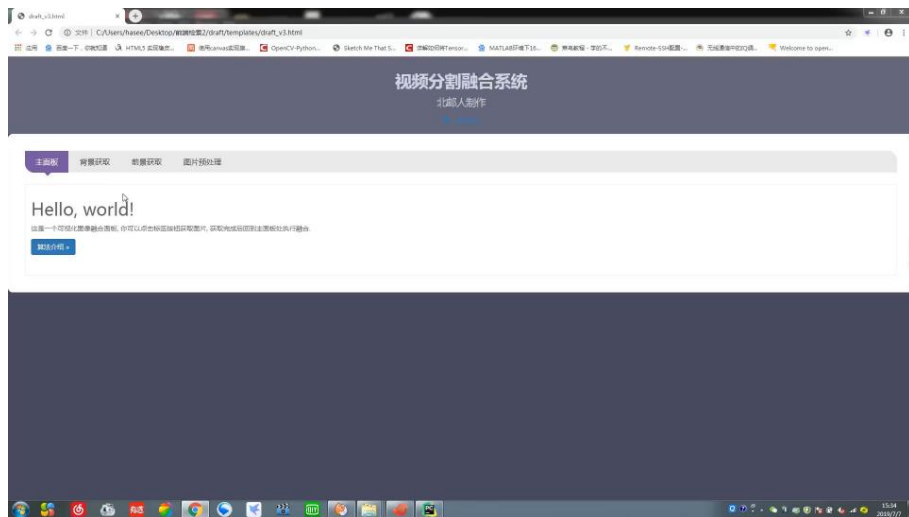
#	Δpub	Team Name	Kernel	Team Members	Score ?	Entries	Last
1	—	JD AI Research - Image			0.03421	94	2d
2	▲1	MTDP-VIC			0.04350	101	2d
3	▼1	EB Lab			0.04370	77	2d
4	—	Hulk			0.04484	58	2d
5	▲1	mmmsf123			0.04660	74	2d
6	▼1	Avengers			0.04719	81	2d
7	▲1	ZXM2019			0.04746	15	2d
8	▼1	MM			0.04798	129	9d

MindCamera: 合成脑海中的影像 实现“所想即所得”的愿望。

- 融合各种最先进的深度学习技术，将图像、视频处理过程简单化、自动化、抛弃复杂的软件，让每个人都可以轻松、高效地制作出高水准的图像视频作品。
- 草图检索，语义理解，智能P图，目标检测分割，风格迁移，视频分割融合等AI技术的**集成创新**！



MindCamera: 视频分割融合演示



- 借助我们在图像领域积累的经验，衍生出了**视频融合算法**
- 智能地检测出每帧视频中的物体，并将其分割出来与背景视频进行融合。
- 产品**主打图片和视频的自动化抠图和组合以及风格迁移**，界面简洁 而高效，力图让每个人都能轻松制作出称心如意的高水准图片和视频。用于**朋友圈晒图+短视频DIY**

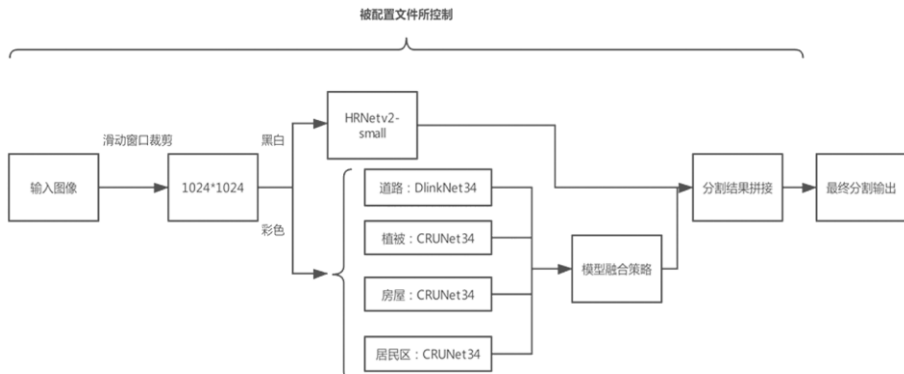
- MindCamera: Interactive Sketch-Based Image Retrieval and Synthesis. **IEEE Access** (2018)

深度学习实现卫星图像分割与识别



- 卫星领域能够产生大量的图像数据，在卫星图像上进行场景特征检测，适合应用深度学习方法研究
- 2019年航空系统装备部第一届“天智杯”卫星图像分割挑战赛，优胜奖&最佳表现奖

算法介绍



算法总体上采用了模型融合的思路，黑白图片使用了高分辨率表征 HRNetV2-small 结合难样本挖掘来进行多分类的分割。彩色图片对每个分割类别定制了对应的模型：道路采用能够捕获大和多尺度感受野的 DlinkNet34，其他类别采用提出的 CRUNet 结构，编码器采用 ResNet-C34，解码器融合了 CARAFE 自适应上采样和 deconv 结构。训练上针对每个类别采用不同的参数，利用 SyncBN 技术使得 BN 层的训练更合理，采用 focalloss 处理正负样本不平衡问题。算法程序易于使用，只需在配置文件中写好对应参数，程序便能一键运行。本算法可以应用在大规模高分辨率卫星图像的精确分割中。

3D裸手交互



操作模式用于和虚拟现实系统进行物理交互，完成抓握、触碰等动作



新的人机界面

采用更自然、更直观的方法与远程电力设备进行通信。例如，对危险区域的设备，手势识别消除了设计按钮的需要。尤其是深度视觉的动作捕捉精度大幅度提升，使得这一新奇的功能有机会实现。



姿态识别

检测并返回图片或视频中的所有手部矩形框位置，以及手势的含义。目前支持识别 19 种常见手势，包括OK、比心、胜利、握拳等。



动作识别

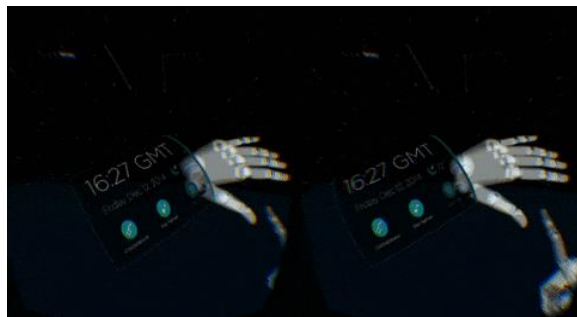
识别 远程对某设备的手势交互，菜单，拖拽，查看等操作。



手势交互应用场景

第一
节

- **VR:** 摆脱手柄, 实现更精细操控。案例: 手部作为菜单:
需要一定的交互设计, 参考
<https://zhuanlan.zhihu.com/p/36742068>
案例: 体验性质的创造性VR活动(文物, 饮食体验, 极限体验) :
https://www.youtube.com/watch?v=GQJQ_kon27g
- **AR:** AR需要大量的手势交互, 菜单, 拖拽, 查看等操作。
- **智能硬件:** 智能开关控制。
- **车载电子:** 结合HUD, 增强驾驶体验:
https://www.youtube.com/watch?time_continue=7&v=eKWVDj4-JG0
- **教育:** 尤其是医疗教育
<https://www.youtube.com/watch?v=CtVZygLd4Y4>
- **虚拟触控:** 家居做饭场景
- **无人机智能识别** : 大疆



制造场景与物体建模与识别

机械拾取和放置

- 在手机制造中可以利用这项操作从托盘或传送带上拾取物品，用于再加工或分拣。从传送带上拾取还需要先进的视觉系统支持。
- 常用物体目标检测、上千类物体识别，准确率95%。
- 基于两项基本操作：1、挑选—成功抓取对象的行为。2、感知—识别和分类对象的能力。
- 在不同场景抓取识别相关物体（手机主板）
- 建立流程与特定物体之间的关联与认知。比如上板、印刷、贴片等



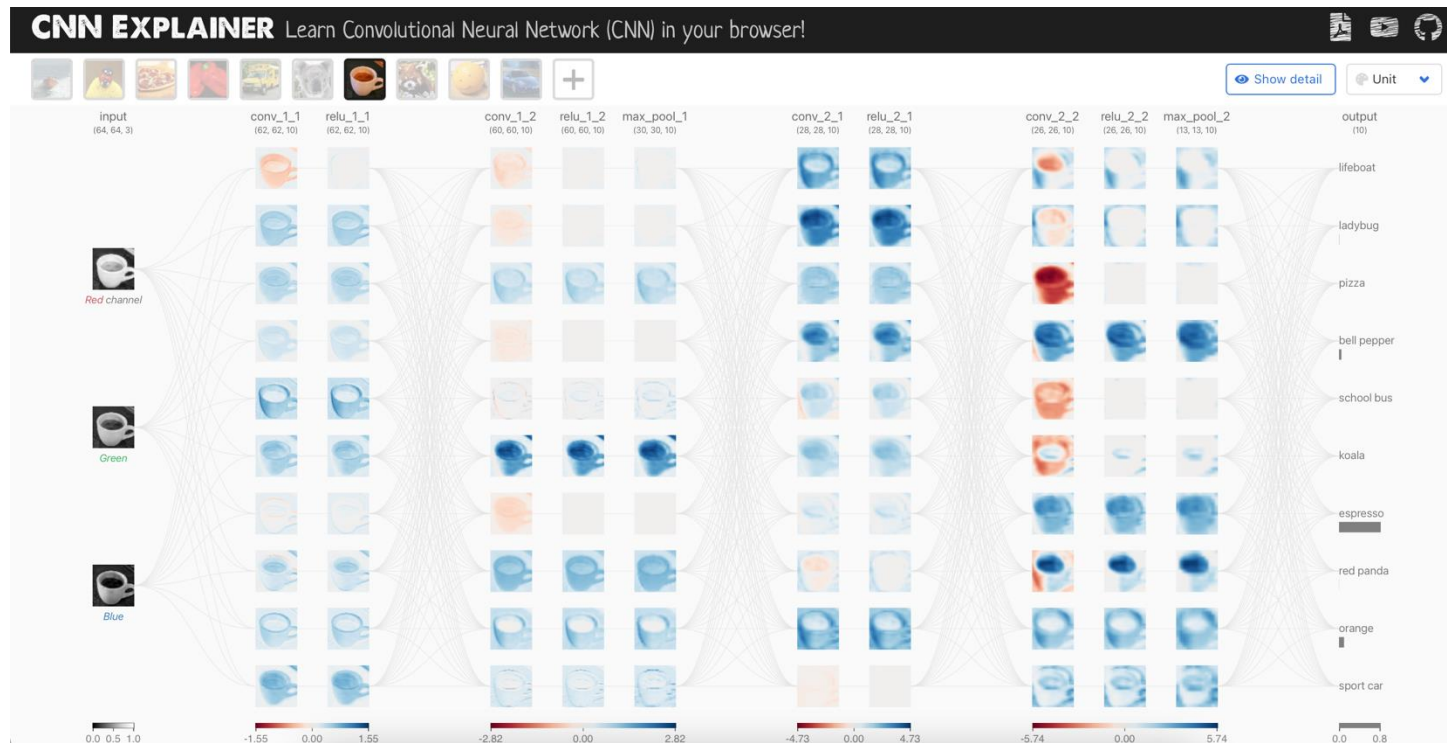
使机器人能够评估一堆随机物体，减少工人的重复劳动，并确定在杂物中抓取电路板等，不必之前了解物体。

- 首先用正确的标识标记图像，然后将给定图像中的像素与给定对象的正确标签（电路板）相关联。
- 比较东西对于人类来说，可能很容易识别，但实际上，作为像素，机器人看起来可能显着同。确保算法适合这些训练样例，需要有足够的训练样例，这样当我们给它一个新的对象时，它就会预测正确的标签。

Toward Learning Automation for Industrial Applicable Object Detection Models from Weakly Labeled Visual Data,
Computers in Industry（二区）

观察和体会一下CNN

<https://poloclub.github.io/cnn-explainer/>



李宏毅老师的《机器学习》

<https://speech.ee.ntu.edu.tw/~hylee/ml/2023-spring.php>

重点关注大模型

ChatGPT

预训练大模型

Diffusion Model